OpenURMA: A Clean-Room Open Implementation of the Unified Bus Protocol

Bojie Li

Pine AI

Modern datacenter RDMA is bottlenecked at the network interface, not the wire. A NIC running RoCE or InfiniBand holds per-connection state for every (application, remote-endpoint) pair — hundreds of megabytes at 1024-application fanout — and pays a four-traversal PCIe round trip on a 64-byte operation, inflating latency an order of magnitude beyond the wire. Both follow from the Queue-Pair-over-PCIe abstraction RDMA inherits from InfiniBand.

Huawei's *Unified Bus* (UB), a public 2025 specification, changes the abstraction: it decouples per-application endpoint state from per-host transport state so connection context grows additively, exposes ordering as opt-in, and reaches remote memory through native CPU load/store to an on-chip-bus controller. UB ships in Huawei's closed Ascend 950 silicon.

OpenURMA is the first clean-room open implementation of UB's transport and transaction layers, realised at three tiers — synthesisable RTL on Alveo U50, a cycle-level two-node SystemC simulator, and a gem5 full-system scaffold — each with a matched OpenRoCE (RoCEv2 RC) baseline. The contribution is the implementation, harness, and controlled comparison closed silicon does not admit. On the canonical 64-byte remote fetch — LOAD on UB-spec §8.3, READ on RoCEv2 RC — UB's load/store path delivers ≈500 ns end-to-end, **4.47**× below the matched baseline (2236 ns), sustains 2.80× higher throughput, and fits in ≈14% of a U50's LUTs.

1. Introduction

A modern datacenter network is two interconnects fused into one. From the application's view, RDMA looks like a load or store to remote memory — bus-like, low-latency, address-and-go. From the NIC's view, every operation is a queued work request shuttled across PCIe to a peripheral device — network-like, with all of the network's per-message machinery and the peripheral's PCIe crossing. For two decades the fusion was tractable: connection counts stayed small, operation sizes stayed large, and the gap between the bus-like view and the network-like implementation could be papered over with cleverer software above the device. AI-training workloads broke both assumptions. A single job now exchanges 64-byte gradient updates across thousands of GPUs; the bus-like view's promises collide with the network-like implementation's costs, and the collision is structural.

The two costs of the collision are both consequences of one design stance: *the NIC is a peripheral*. Peripherals communicate with the CPU through a paired-queue programming model, so the NIC must hold per-pair connection state; peripherals sit behind PCIe, so every operation crosses the PCIe boundary. Under RoCEv2 RC, each (local application, remote endpoint) pair is a Queue Pair (QP) carrying ∼512 B of NIC state; at 1024 of each, the NIC's working set is ∼537 MB, past any commodity NIC's on-chip SRAM and into the regime where every operation pays a PCIe round trip to refetch its QP [24, 20, 55]. The same operation pays a second cost even when state stays on chip: a 64-byte READ spends roughly 1.5 μs of its ∼2 μs round trip on four PCIe traversals (doorbell MMIO, work-queue DMA fetch, completion DMA write, CPU poll-miss across PCIe coherence), while the wire delivers in ∼200 ns. Software cannot move the peripheral; hardware inside the peripheral cannot move PCIe. What removes both costs is moving the peripheral.

Huawei's *Unified Bus* (UB) [17] is a 2025 specification that moves the peripheral. The NIC is no longer behind PCIe but on the on-chip bus, addressed by the CPU through ordinary load and store instructions; connection state is no longer per-application-pair but per-host-pair; ordering is no longer always-on but opt-in. Huawei's Ascend 950 NPU (neural processing unit) [18] ships UB at commodity scale, but the silicon is closed and the spec is unaccompanied by a public implementation that researchers can measure, instrument, or prototype against.

The three architectural moves UB makes are not independent choices; they form a chain in which each move makes the next possible (Figure 1).

1. **Split the transport layer.** The Queue Pair fused application identity with transport reliability into one object, so state had to scale with the product of the two counts. UB separates the two: per-application endpoint state lives on a *Jetty*; per-remote-host transport state lives on a *TP Channel*. Per-NIC state then grows additively in the number of local endpoints N (one per thread at the high-performance design point, since threads sharing a work queue serialize on it) and remote hosts M,[1] as $O(N{+}M)$ instead of $O(N{\cdot}M)$. Bounding state is what makes (2) possible.

[1] M counts remote hosts — the conservative one-endpoint-per-host case. RC binds each QP to a specific remote endpoint, so a host exposing P addressable endpoints costs RoCE $N{\cdot}M{\cdot}P$ QPs; UB's TP Channel is per-host regardless of P, because the remote application is named in the packet header rather than bound into the connection.

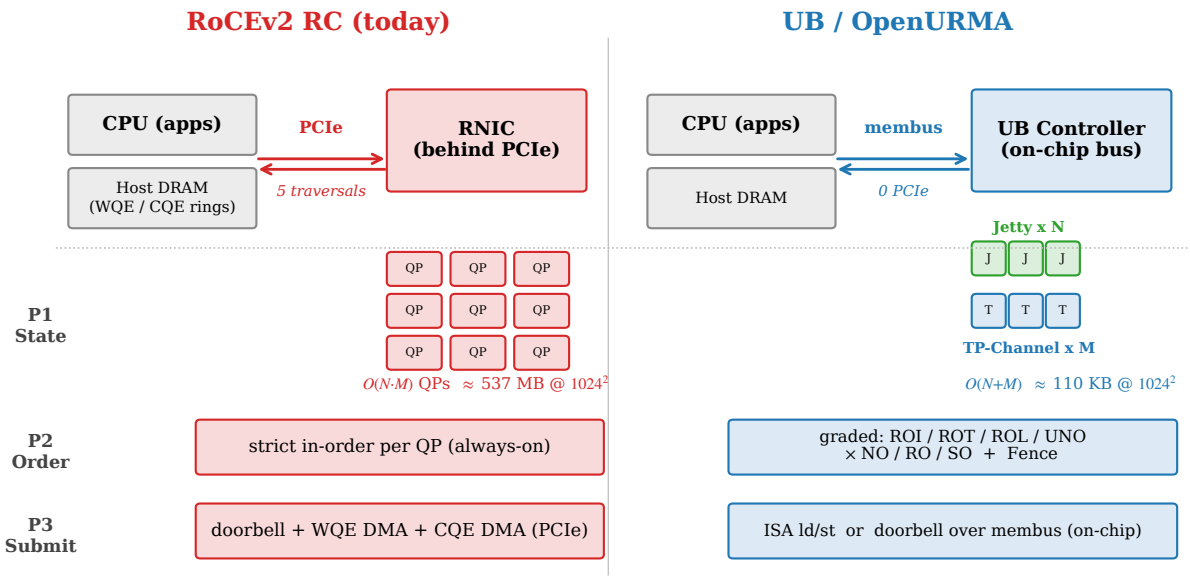


Figure 1: The three architectural moves and their dependencies. RoCEv2 RC (top) puts the NIC behind PCIe, holds one Queue Pair per (application, remote-endpoint) pair, and enforces strict order on every operation. UB (bottom) splits state along the transaction/transport line so it grows as $O(N+M)$; places the controller on the on-chip bus, where the CPU can reach it with loads and stores; and exposes ordering as four opt-in axes that reuse counters the layer split already provisions. The arrows mark the chain: bounded state is what lets the controller live on-bus; the on-bus controller is what lets the load/store path exist; the layer split is what makes ordering cheap.

2. **Move the controller onto the on-chip bus.** A NIC whose working set fits in on-chip SRAM can live next to the CPU on the on-chip bus rather than behind PCIe; a NIC whose working set spills to host DRAM cannot, because each spill becomes a host-memory access. Putting the controller on-bus is what makes (3) possible.
3. **Admit a load/store data path.** Once the controller is on the on-chip bus, a CPU's load or store instruction can reach it directly. The four PCIe traversals collapse into a single on-chip-bus crossing, and the work-queue and completion-queue machinery elides for small synchronous operations.

A fourth move — opt-in ordering — rides on the same per-application counters the first move provisions, so it costs zero extra pipeline cycles on operations that do not request gating, while letting workloads that need a barrier pay only for that barrier.

OpenURMA is a clean-room open implementation of UB's transport and transaction layers, realised at three modeling tiers — RTL on Alveo U50, a two-node SystemC simulator that models both endpoints of the wire, and a gem5 full-system scaffold running real ARM binaries against the SystemC NIC — each against a matched *OpenRoCE* (RoCEv2 RC) baseline on the same toolchain and harness, so both stacks' numbers come from identical modeling assumptions rather than published vendor data. On the canonical 64-byte remote fetch, the load/store path delivers ≈500 ns end-to-end — **4.47**× below the RoCEv2 baseline (2236 ns) for the same CPU-fetches-64 B operation.

Contributions.

- **First clean-room open implementation of the Unified Bus protocol.** Synthesisable RTL for the transport and transaction layers, closing 322 MHz post-route at roughly 14% of an Alveo U50's LUT budget.
- **Apples-to-apples OpenRoCE baseline.** A RoCEv2 RC implementation on the same toolchain, target, and harness.
- **Multi-tier evaluation infrastructure.** A cycle-level two-node SystemC simulator that accounts for both endpoints of the wire, plus a gem5 full-system scaffold that puts a CPU in the loop; jointly necessary to measure the end-to-end CPU-to-remote-DRAM path UB claims to shorten.
- **Cross-cutting validation,** reproducing published ConnectX-7 RDMA WRITE latency within ±5%, congestion-control dynamics against DCQCN, selective-vs-Go-Back-N loss recovery, and a YCSB-A application port.

OpenURMA is released at `https://github.com/bojieli/OpenURMA`.

Roadmap. §2 places UB in context against the peripheral-NIC abstraction and three decades of patches around it. §3 walks the design choices and §4 the pipeline decomposition and timing closure. §5 sets up the evaluation; §6 establishes that the NIC synthesises, is cheap, characterises its raw latency, and validates the model against ConnectX-7. The evaluation is then organised by design commitment. The four moves of §2.4 reduce to three measurable commitments: bounded state (Move 1), the load/store latency collapse (Moves 2 and 3 jointly — the on-bus controller's payoff *is* the latency at which a load/store reaches remote memory, so the two are measured as one number), and opt-in ordering (Move 4). §7 validates bounded state, §8 the load/store latency collapse, and §9 near-free opt-in ordering; §10 confirms all three under a real OS. §11 then covers reliable transport under

loss and congestion, and §12 the scale-out reach coherent fabrics cannot match, with §13 summarising. §14 discusses limitations; §15 surveys related work.

2. Background

This section grounds the introduction's thesis in concrete mechanics: what makes the NIC a peripheral, why the peripheral stance imposes both costs by construction, why three decades of fixes around it leave the abstraction in place, and what Unified Bus replaces it with.

2.1 The peripheral-NIC abstraction

Every commodity RDMA NIC — InfiniBand HCAs, Mellanox/NVIDIA ConnectX, Broadcom Thor, Intel IPU — is a PCIe peripheral. The CPU sees it through a memory-mapped BAR; the NIC sees the CPU through DMA. The pair communicates by a Queue-Pair protocol: the CPU posts work-queue entries to host memory, signals via MMIO doorbells, and the NIC consumes them by DMA and writes completions back the same way. This programming model is itself a small network protocol spoken across PCIe, and the model fixes two properties of the system before the wire is even reached.

The QP fuses what should be two layers. A Queue Pair is named by a (local-application, remote-endpoint) pair and carries both transaction-layer state (work queues, permissions, the identity of *who is calling*) and transport-layer state (sequence numbers, retransmit window, RTO timer, congestion state — the bookkeeping of *how packets are delivered*). Textbook protocol design separates these layers; the QP collapses them into one object whose unit of accounting is the application pair, not the application count or the peer count. Total NIC state therefore grows as $O(N{\cdot}M)$. At $(N, M){=}(1024, 1024)$, $\sim$1 M QPs at $\sim$512 B each is $\sim$537 MB, beyond any commodity NIC's on-chip SRAM. When the working set spills to host DRAM, every operation pays a PCIe round trip to refetch its QP before it can even consult its sequence number [24, 55].

The PCIe attachment imposes four traversals per operation. Because the NIC is a peripheral, every operation crosses the PCIe boundary four times. On a 64-byte READ: the CPU writes a work-queue entry to host DRAM and then a doorbell to the NIC's MMIO BAR (traversal 1); the NIC DMA-reads the work-queue entry (traversal 2); the NIC emits the wire request, receives the response, and DMA-writes a completion entry to host DRAM (traversal 3); the CPU polls the completion queue and incurs a PCIe-coherence miss to fetch the freshly written line (traversal 4). Of the $\sim$2 μs round-trip time, $\sim$1.5 μs is consumed by these four traversals; the wire itself delivers in $\sim$200 ns. The traversals are not an implementation accident; they are what the model requires when the two communicating parties live in disjoint address spaces.

Both costs follow from one stance. Fixing either in isolation leaves the other in place: connection sharing (XRC, the eXtended Reliable Connection, and SRQ, the Shared Receive Queue) reduces state but keeps the four PCIe traversals; doorbell coalescing and inline work-queue-entry (WQE) shortcuts trim one traversal but keep the QP fusion. The abstraction is what limits the optimisation budget.

2.2 Why prior fixes are patches

Three decades of work on RDMA scale fall in three buckets, none of which removes both costs. *Software above the device* — FaSST [20], 1RMA [50], Snap's Pony Express [41], the broader SmartNIC-pacing literature — works above the verb interface; it can change the application's view but not the peripheral attachment or the programming model. *Hardware inside the device* — SRNIC [55], StaR [54], the IRN loss-recovery line [43, 39, 15, 34], MP-RDMA [40], ConWeave [52] — rebuilds the NIC's internals symptom by symptom, but the NIC remains a PCIe peripheral with a Queue-Pair programming model. *Programmable substrates* — Tonic [3], NanoTransport [19], StRoM [49] — host new transports on FPGA or P4 fabric, but they are substrates, not new transports.

2.3 Unified Bus: the spec, the silicon, the gap

UB is the first RDMA-class specification since RoCEv2 (2014) to replace the abstraction rather than patch it. The 2025 spec spans physical through verb layers; this paper engages only transport and transaction, where the structural choices live. The protocol ships in Huawei's Ascend 950 NPU [18] with both the work-queue and load/store data paths as on-die blocks alongside the AI accelerators. The user-space verb library and kernel driver are open-source; the protocol implementation is closed. OpenURMA fills that gap with a clean-room implementation built from the public specification.

2.4 Making the NIC a peer of the CPU

UB stops treating the NIC as a peripheral the CPU programs and starts treating it as a peer the CPU shares an on-chip bus with (Figure 2). This section gives the spec-level mechanism behind each move §1 introduced — the same enabling chain (bounded state $\rightarrow$ on-bus controller $\rightarrow$ load/store path, with opt-in ordering riding on the first), now with the field-level detail.

Move 1: split transaction from transport. The QP fused two layers into one object, so state had to scale with their product. UB separates them: a *Jetty*[2] holds

[2]The name marks the break from the connection. A *jetty* is a wharf: one structure at which many vessels berth in turn, sharing the harbour rather than each dredging a private channel to shore. The metaphor is deliberate — a Jetty is an application's berth at the network, not a wire to one peer, and the requests it issues occupy that berth only while each is in flight. A new abstraction wants a word that does not smuggle in the old one's assumptions: "connection" and "queue pair" both presume a

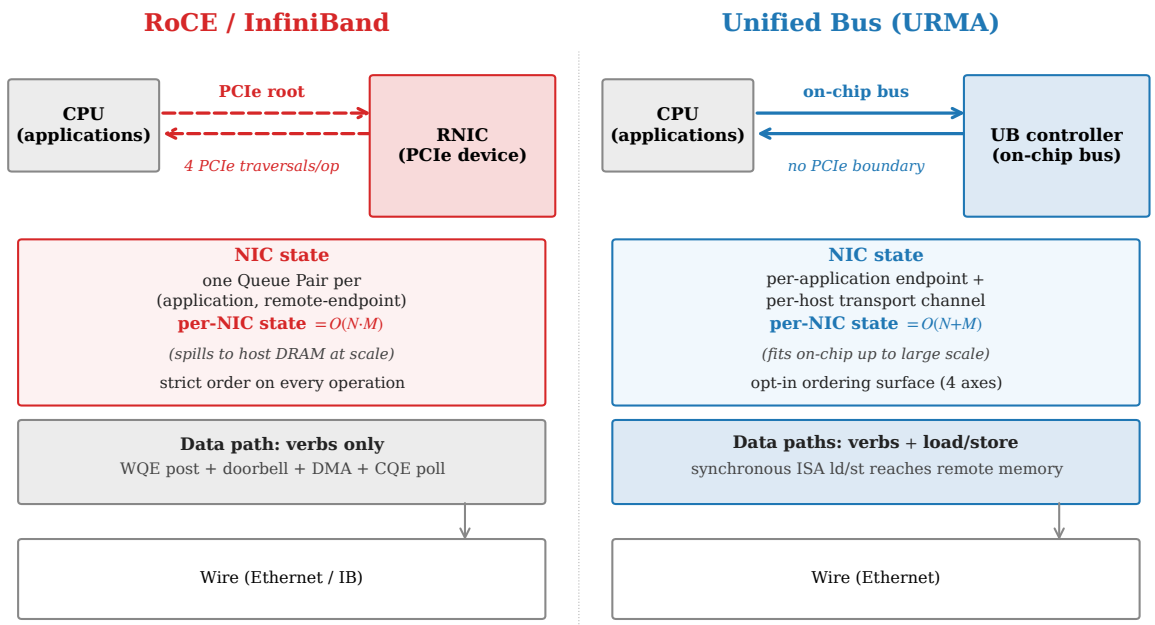


Figure 2: Architectural comparison. RoCE puts the NIC behind PCIe; it holds one Queue Pair per (application, remote-endpoint) pair, enforces strict order on every operation, and admits only the verb-driven data path. Unified Bus puts the controller on the on-chip bus; it holds per-application endpoint state separately from per-remote-host transport state, exposes ordering as an opt-in surface, and admits CPU load/store as a synchronous data path alongside the verb path.

the transaction-layer state for one local application (completion-queue handle, access token, and type/state flags, ∼20 B); a *TP Channel* holds the transport-layer state for one remote host (sequence-number windows, SACK bitmap, congestion state, ∼56 B) (Figure 3). Any Jetty addresses any remote endpoint through the shared per-host TP Channel pool, with the destination carried in the packet header rather than baked into a per-pair binding. State grows as $O(N{+}M)$: at $(N, M){=}(1024, 1024)$, the Jetty, TP Channel, and shared memory-region tables together hold ∼110 KB instead of 537 MB (§7).

The split pays for itself with one extra lookup per outbound packet (which TP Channel by destination host) and a protocol surface that exposes Jetty and TP Channel as separate first-class objects with separate lifecycles. The QP's one-object-per-peer simplicity is the cost of admission. The split bounds the cross-product, not per-endpoint contention: a Jetty carries its own work queue and doorbell, so sharing one across threads serializes exactly as a shared QP does. N therefore counts independent issue contexts identically for both stacks; what UB removes is the M-fold multiplication of that count, not the per-endpoint serialization.

Move 2: put the controller on the on-chip bus. At $O(N{+}M)$ state the connection working set fits in on-chip SRAM at scales that previously forced spill to host DRAM, so the controller can live on the CPU's on-chip bus rather than behind PCIe: the CPU reaches it through ordinary memory-mapped regions and the PCIe boundary collapses into a single bus crossing. The cost is that the controller must sit on the CPU's bus segment.

bound peer, which is exactly what the layer split removes.

Move 3: admit a load/store data path. With the controller on-bus, a CPU load or store reaches an address aperture it owns (Figure 4); the controller turns each instruction into one wire transaction — no work-queue entry, doorbell, DMA, or separate completion — and the load blocks until the response returns. Only short synchronous operations (loads, stores, same-path atomics) are admissible; bulk and asynchronous traffic stays on the work-queue path. The two complement rather than compete — load/store wins on short, latency-tight operations, the work-queue path on long, throughput-bound ones.

Move 4: make ordering opt-in. This move sits outside the chain but on top of Move 1. RoCE RC enforces strict in-order delivery on every operation; UB exposes ordering as four orthogonal axes the application opts into per channel and per operation. The *service mode* controls how aggressively packets may reorder on the wire (relaxed admits multi-path spreading; strict forces single-path delivery); the *execution order* controls whether the issue stage may emit before the previous operation commits; the *fence* is an explicit application barrier; the *completion order* bit controls whether completions retire in arrival or issue order. A pure-fanout broadcast bypasses every gate; a barrier-after-collective pays exactly the gating its workload needs.

Opt-in ordering is cheap, robust, and semantically apt at once. *Cheap:* the gating indexes per-Jetty counters Move 1 already provisions, so it adds zero pipeline cycles on operations that bypass it. *Robust:* strict total order is fragile under failure — a NIC promised in-order delivery cannot retire any completion while an earlier operation stalls, so one slow path head-of-line-blocks every application on the channel. *Apt:* collective and gradient-

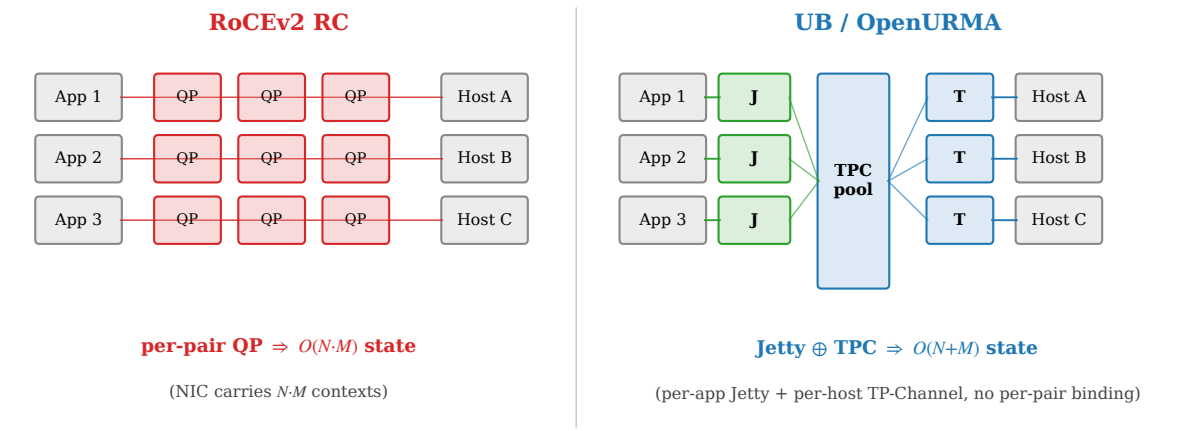


Figure 3: State models compared. RoCE binds one Queue Pair to every (application, remote-host) pair, so per-NIC state grows as $O(N{\cdot}M)$. UB decouples per-application state (Jetty) from per-host transport state (TP Channel); any Jetty addresses any remote endpoint through the shared TP Channel pool, and per-NIC state grows as $O(N{+}M)$.

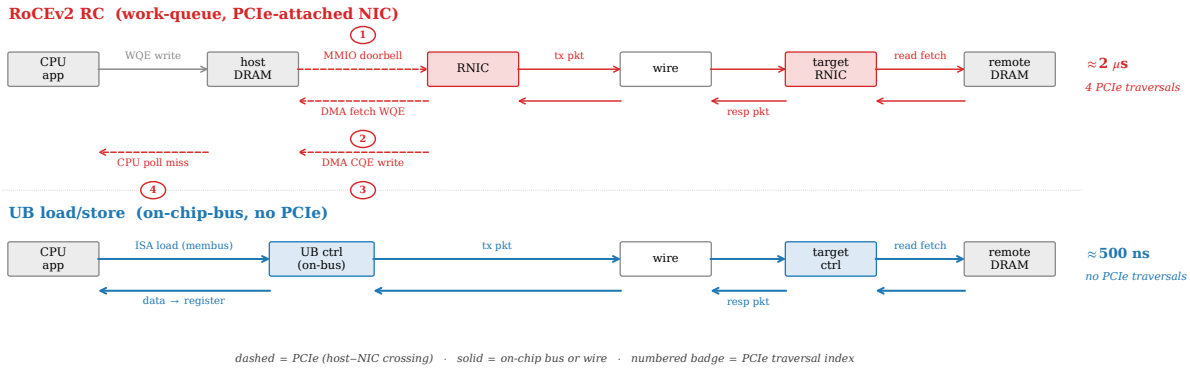


Figure 4: Per-operation data path for a small synchronous read. The traditional work-queue-driven path (top) traverses four PCIe crossings — doorbell over MMIO, work-queue-entry DMA fetch, completion-entry DMA write, and a CPU poll-miss across PCIe coherence. The Unified Bus load/store path (bottom) replaces all four with on-chip-bus crossings: a CPU load instruction reaches the controller, the controller emits the wire packet, the response arrives, and the data lands directly in the CPU register that issued the load.

exchange traffic is order-insensitive by construction (the reduction is commutative; the application resynchronises at the barrier), so strict per-operation order spends reliability machinery on a guarantee the workload never asked for. The opt-in surface lets each workload pay only the gating it needs.

2.5 The OpenClickNP toolchain

OpenURMA is built on OpenClickNP [27], a clean-room re-implementation of the ClickNP element model [30]. The model is a familiar one: a NIC pipeline is a directed graph of *elements*, each with typed input and output ports, local state, and a per-cycle handler. The toolchain lowers a single source description through three back-ends: a software emulator for unit testing, a cycle-accurate SystemC simulator for performance measurement, and a Vitis HLS back-end that emits synthesisable C++ for Vivado synthesis on the Alveo U50. The same source produces all three artifacts. The relevance to this paper is twofold: the model makes element-level cost (LUTs, pipeline-II, BRAM) directly measurable, and the three back-ends are how OpenURMA produces both the RTL tier and the SystemC tier from one description.

3. Design

This section describes how OpenURMA realises Unified Bus transaction- and transport-layer protocol in RTL. We organise the description around the three architectural commitments the four moves of §2.4 reduce to bounded state (Move 1), opt-in ordering (Move 4), and the load/store data path (the on-bus controller of Move 2 together with the load/store path of Move 3).

3.1 Realising bounded state

The Jetty / TP Channel split maps onto two pipeline regions. The transmit path consults a per-Jetty table to admit and gate the work request, then hands it to a per-host TP Channel scheduler that attaches the sequence number, picks the multi-path lane, and queues the packet. The receive path inverts the flow: per-TP-Channel logic validates and reorders by sequence number, then a per-

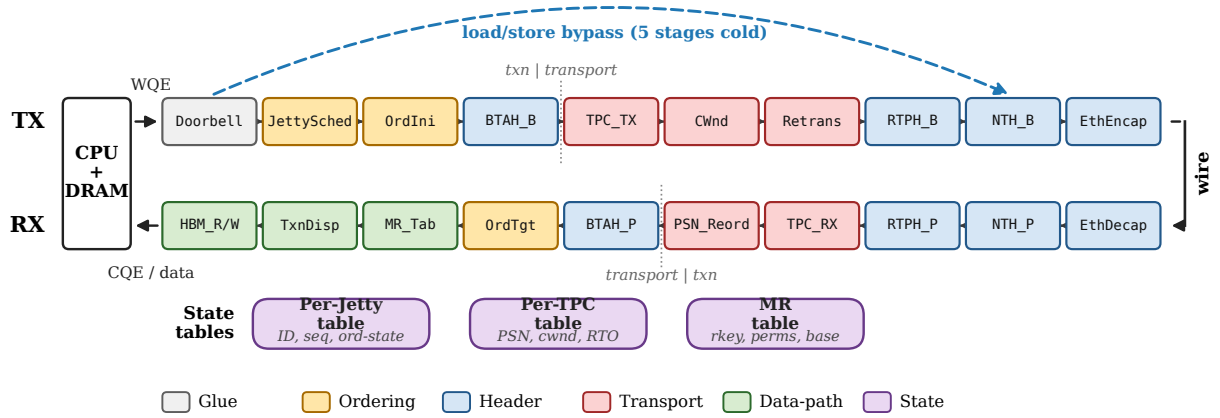


Figure 5: OpenURMA's NIC as a ClickNP element graph. The TX path (top) flows from CPU doorbell to wire; the RX path (bottom) inverts. Each box is one element, coloured by its functional category (legend at bottom): glue, ordering, header parse/build, transport, on-NIC data path, and state. The dotted vertical markers separate the transaction layer (application-level Jetty scheduling, ordering, memory-region checks, atomic operations, completion generation) from the transport layer (per-host sequencing, retransmission, congestion control). The dashed arc shows the load/store data path (§3.3): a CPU load or store reaches the controller and the bypass element produces a wire packet in five stages cold, skipping the ordering and transport-state machinery entirely.

Jetty table maps the destination header to the right receive queue. Two tables, two indices, two scaling regimes. The pipeline pays one extra header field per packet (the destination Jetty id) and one extra lookup per side of the wire; in exchange, nothing in the pipeline binds a transmit context to a receive context, so the protocol's $O(N+M)$ scaling becomes the pipeline's. At $(N, M)=(1024, 1024)$ the per-NIC connection-state working set is ∼110 KB versus ∼537 MB for the matched RoCE QP table (§7).

3.2 Realising opt-in ordering

The four ordering axes become four gating stages on the pipeline. Each decides at one well-defined point whether the operation in flight may proceed; together they form a mesh that adds zero pipeline cycles to operations whose mode bypasses every gate. The trick is in what the gating logic indexes against.

Reused counters. Each gate reads the same three inputs: a per-Jetty sequence counter ("do prior operations from this application still need to complete?"), a per-channel outstanding-window counter ("can the wire absorb another?"), and a per-Jetty fence latch. The state model in §3.1 already holds all three — the spec defines the relevant sequence numbers as per-Jetty objects, and the per-Jetty table carries them regardless. The ordering surface reads from counters the state model writes to; it does not add a parallel bookkeeping structure.

Two reorder buffers, not one. A subtler choice is to carry *two* reorder buffers serving disjoint contracts (Figure 6). One acts at the transport layer on packet sequence numbers and delivers byte-correct flits regardless of any application's ordering choice, so a TP Channel can multiplex multiple applications without one application's wire-byte gaps stalling another's. The other acts at the transaction layer on per-Jetty sequence numbers and delivers in-issue-order completions regardless of wire-byte order, so the transmit path can spread packets across multiple network paths without breaking the application's view. Collapsing the two would couple the contracts: every multi-path packet gap would block the completion stream; every per-application dependency stall would back-pressure the wire. The separation is what makes unordered service mode and strict mode correctness-safe at the same time. §15 contrasts this with the per-transaction bitmap approach of the Ultra Ethernet Consortium (UEC) and MP-RDMA.

3.3 Realising the load/store path

The work-queue-driven path runs ten pipeline stages cold (work-queue scheduling, ordering, header build, transport accounting, congestion check, multi-path lane select, retransmit-buffer write, framing, transmit). For a small synchronous operation, every one is overhead the abstraction does not need.

The load/store path drops them. A CPU `ld` or `st` to the controller's address aperture builds the transaction header, attaches a transport-bypass flag, builds the network header, and frames the packet for the wire — five stages cold. No work-queue entry, no completion entry, no sequence-number allocation (the bypass flag tells the receiving NIC to skip its transport-layer state machine), no retransmit-buffer slot (the CPU re-issues the load on timeout, which is cheaper than transport-layer reliability on a path that is synchronous anyway). The path admits only operations whose semantics survive transport bypass — small loads, stores, and same-path atomics — and the controller must sit on the CPU's on-chip-bus segment; bulk and asynchronous transfers stay on the work-queue path.

3.4 Hardware fanout: target-side dispatch

Bounded state has a second consequence: target-side demultiplexing — which local application receives an incoming SEND — can live in hardware rather than in

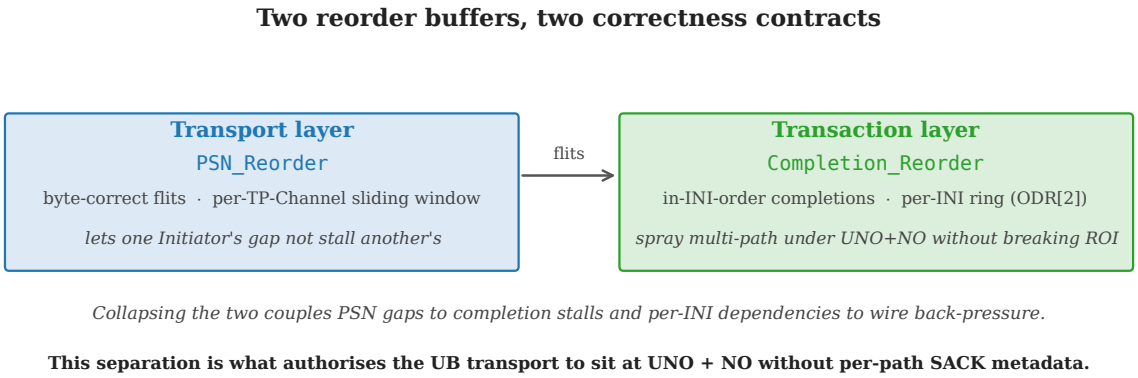


Figure 6: The pipeline carries two reorder buffers serving disjoint correctness contracts. Packet-sequence reordering at the transport layer delivers byte-correct flits regardless of application ordering; transaction-sequence reordering at the transaction layer delivers in-issue-order completions regardless of wire-byte order. The separation is what makes multi-path spreading and opt-in ordering composable.

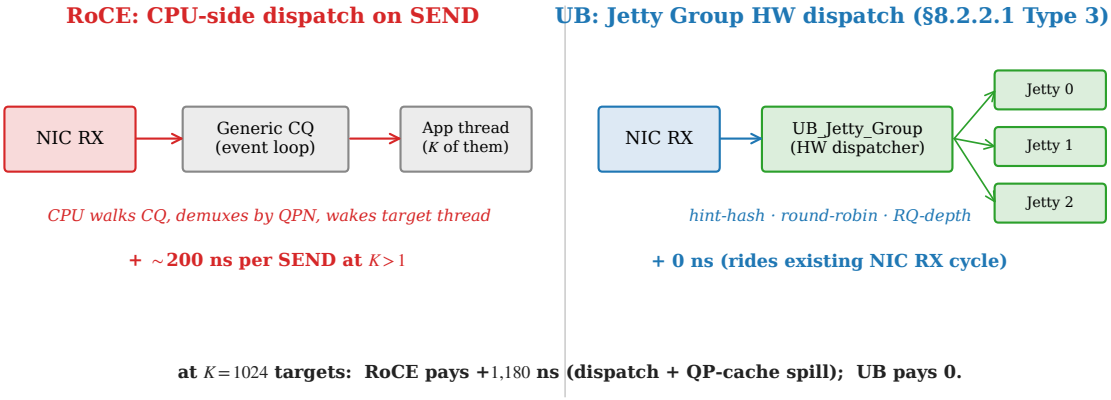


Figure 7: Target-side SEND dispatch. RoCE demultiplexes through a shared completion queue plus an application event loop; UB's Jetty Group performs the dispatch in hardware on the membus crossing already paid for by NIC RX.

a CPU event loop. RoCE handles this in software (the target NIC writes a completion to a shared queue and the application event loop dispatches at CPU speed). UB defines a *Jetty Group* in which a single externally visible identifier represents a set of member Jetties; the receiving NIC dispatches each incoming SEND to a member in hardware under one of three policies (hash on an initiator-supplied key, round-robin, or receive-queue-depth load balancing) (Figure 7). The target CPU is no longer on the SEND fast path. The cost: one extra RX pipeline element and ∼80 B per 8-member group. §7.3 measures the saving at ∼200 ns per SEND for small fanout, growing to ∼1200 ns once RoCE's QP spill compounds with the CPU dispatch.

3.5 What the implementation makes visible

The implementation puts numbers on the dependencies the introduction asserted. The on-chip controller needs bounded state: with per-Jetty and per-TP-Channel tables fitting in ∼110 KB at $(1024, 1024)$, every load/store reaches the relevant context in on-chip BRAM; were the state to spill to host DRAM, the controller would pay a host-memory access on every operation and lose its latency edge. Opt-in ordering needs the layer split: the gating logic indexes per-Jetty counters that the table already provisions, costing ∼13 KLUT and zero pipeline cycles on operations that bypass gating; a QP-centric design would have to add those counters from scratch on every operation. Multi-path spreading needs the two-reorder-buffer split: the transport- and transaction-layer reorder buffers (§3.2) carry disjoint contracts, so an unordered application receives unordered completions while an ordered application sharing the same wire receives ordered ones; collapsing the buffers would force one contract onto both, and the unordered mode could no longer authorise spreading. The next sections quantify what the resulting design costs (§4) and what it saves (§7 onwards).

4. Implementation

OpenURMA is ∼3.5 KLOC of element-source across 39 pipeline elements spanning the transport and transaction layers; the parallel OpenRoCE baseline (RoCEv2 RC) is ∼1.3 KLOC across 21 elements. Both sit on the same OpenClickNP toolchain [27], so every comparison is between artifacts from an identical lowering pipeline. We

describe each piece by what it does, not what it is named.

4.1 Pipeline decomposition

The 39 elements break into six categories whose counts the protocol fixes. **Ten header parser/builder pairs** handle the four layered headers (Ethernet framing, network-transport, reliable- and unreliable-transport, and base-transaction). **Nine transport-layer elements** implement the per-host TP Channel pipelines, the retransmission and packet-sequence reorder buffers, the per-packet and transaction-level acknowledgement generators, the congestion-control window and its echo-feedback element, and the multi-path dispatcher. **Four ordering elements** realise the gating surface of §3.2: the Jetty scheduler, a completion reorder buffer, and initiator- and target-side order trackers (two because the gating responsibility shifts with the service mode). **Seven data-path elements** cover memory read/write, atomics, per-Jetty receive, the target-side dispatcher, the opcode router, and the completion generator. **Three state tables** hold memory-region permissions, per-Jetty state, and per-TP-Channel state. **Six glue elements** provide the doorbell, dispatch multiplexers, completion-notification tees, completion-queue streamers, and the retransmission RTO timer.

Two decompositions carry design intent: the initiator- and target-side order trackers are separate because gating shifts between endpoints with the service mode (relaxed at the target, strict at the initiator), and the completion reorder buffer is distinct from the packet-sequence reorder buffer because the two serve disjoint correctness contracts on disjoint sequence-number spaces (§3.2). The work-queue transmit path threads ten stages cold, doorbell to Ethernet encapsulation; the receive path inverts it (Figure 5).

Two elements are new. The *load/store bypass engine* (§3.3) is a separate topology variant that takes a load/store doorbell off the on-chip bus, allocates a one-shot context (no sequence number, no retransmission slot), stamps a transport-bypass flag, and frames the packet — its first wire flit emerges at **8 cycles (≈25 ns)**, 16 below the work-queue path. The *target-side dispatcher* (§3.4) rewrites a SEND addressed to a Jetty-Group identifier to a member Jetty (hint-hash, round-robin, or queue-depth balancing; unregistered identifiers pass through), at ∼80 B per 8-member group.

4.2 Retransmission and congestion control

The retransmission buffer is a 64-slot per-TP-Channel ring supporting both Go-Back-N and selective retransmit; the selective path uses the spec's selective-ack opcode and a per-packet-sequence-number (PSN) bitmap, replayed on an explicit NAK or the RTO timer. Each slot holds one metadata/extension flit pair — enough for control-plane ops and ≤8-byte Writes; multi-flit Write replay (a payload-flit list per slot) runs on the loss-free path but is not yet in the buffer (§14). Congestion control pairs a host-side switch model (queue watermarks stamping the FECN bit) with a per-channel additive-increase / multiplicative-decrease (AIMD) window; in a 200-WR closed-loop test, 9 of 25 packets are marked and the sender's window backs off from 65,536 B to 4,096 B.

4.3 Toolchain and back-ends

OpenClickNP lowers each element from one source through three back-ends: a thread-and-FIFO software emulator (correctness tests, §6.4); a cycle-accurate SystemC simulator on a 1 ns clock matching the 322 MHz target, so cycle counts compare directly to post-route timing (§6.3); and a Vitis HLS back-end that Vivado places and routes (§6.1). An element that closes timing in HLS is exactly the one the emulator tested and the simulator benchmarked. Our only framework change was two element-source pragmas — one to declare an element's initiation interval explicitly, one to forward arbitrary HLS pragmas — needed because the back-end previously hard-coded II=1, over-pipelining six elements whose data dependencies cannot be unrolled. Both are upstream and reused by both stacks.

4.4 Two-node SystemC simulator

The two-node simulator links each NIC stack as a static SystemC library and connects two instances through an EtherLink module with configurable bandwidth and delay. Each endpoint is a full system — host CPU posting doorbells, the NIC TX/RX pipelines, a DDR4 model, a write-back / write-through / uncached cache hierarchy, and the completion-queue write-back path — and the harness sweeps 388 configurations across the six verb categories, four cache policies, 8 B–4 KB payloads, and 100 ns–10 μs link delays.

4.5 gem5 full-system scaffold

For the CPU-to-remote-DRAM end-to-end claim we embed the SystemC NIC into gem5 [38] as a memory-mapped device, so an ARM CPU running Linux 4.14, the `uburma.ko` driver, and a libc-linked benchmark drive the cycle-accurate pipeline through a real OS stack rather than a synthetic injector. The integration took three non-obvious steps — rebuilding the SC libraries against gem5's embedded (ABI-incompatible) SystemC, carving the NIC aperture out of system memory so gem5's crossbar can route it, and draining the TLM pipeline synchronously on each transaction since the atomic CPU never yields to the SC event queue — but none touch the protocol. Bringing the full WRITE → TAACK (transaction-layer acknowledgement) → CQE roundtrip up end-to-end also surfaced three latent pipeline bugs the synthetic-injector tests had masked: a dropped service-mode field, a stranded reorder-buffer flit, and an unrouted received-ACK port. Fixing them is what lets the scaffold report the three completion-queue-entry (CQE) delivery paths of §5.

4.6 Timing-closure sweep

The initial Vitis-HLS pass on the 38 work-queue-driven elements left 8 needing remediation: 5 missed 322 MHz in Vivado P&R and 3 failed at HLS before reaching P&R. The fixes combined the new II/pragma extensions with a few algorithmic redesigns (Table 1).

Element function	Before → After (ns WNS)	Action
Atomic operations	$-1.871 \rightarrow +0.298$	II=4 + mem partition
Completion reorder	HLS-stuck → $+0.672$	head-ring + II=2
Initiator order tracker	$-5.382 \rightarrow +0.318$	II=2
Jetty scheduler	HLS-aborted → $+0.546$	II=2
Target order tracker	$-2.25 \rightarrow +0.101$	II=2 + drain reorder
On-NIC memory write	$-0.628 \rightarrow +0.628$	memory partition
Memory-region table	failing → $+0.729$	shrink to 64 entries
On-NIC memory read	$-0.449 \rightarrow +0.282$	word-sized array

Table 1: Timing-closure remediation, by element function.

The instructive case. The on-NIC memory-read element first backed its 64 KB path with a byte-array plus cyclic partition for parallel byte-bank access. HLS estimated 490 MHz, but Vivado P&R missed by 0.449 ns: the critical path was 8 LUT levels of barrel-shift mux into the partitioned BRAM, routing-bound, and no II or partition factor moved it. The fix was at the data-layout level — an 8-byte-word array indexed by a shifted offset turns each aligned read into one BRAM word access, eliminating the mux and closing at $+0.282$ ns (the same change applied to OpenRoCE). The lesson: HLS's per-element timing estimate is optimistic, routing-bound paths only appear in P&R, and the fix is often in data layout, not pragmas.

5. Evaluation methodology

The evaluation is organised around the three design commitments. §6 establishes that the NIC is synthesizable, cheap, correct, and faithfully modeled; §7–§9 validate each commitment in turn (bounded state, the load/store latency collapse, and near-free opt-in ordering); §10 confirms all three under a real OS; and §11 and §12 cover reliable transport and the scale-out reach that coherent fabrics cannot match. This section first describes the measurement substrate the rest of the evaluation shares.

5.1 Three modeling tiers and a matched baseline

Without physical silicon, every number comes from one of three back-ends lowered from the *same* element source (§4.3), so a kernel that passes correctness is the one that is benchmarked and synthesised: **(1)** a thread-and-FIFO software emulator answers correctness questions; **(2)** a cycle-accurate SystemC simulator (1 ns clock matching 322 MHz) reports latency, throughput, and ordering cost, either standalone per element or wired into a two-node model whose analytical surrounds (host CPU, PCIe/membus, wire, DRAM) close the end-to-end path; and **(3)** a gem5 full-system scaffold runs Linux + the uburma driver + a real benchmark against the SystemC NIC. A matched **OpenRoCE** (RoCEv2 RC) baseline is lowered through the same toolchain at every tier, so all comparisons hold every variable below the protocol fixed. Every figure is reproducible from one CSV-emitting harness.

Four *configurations* appear as columns throughout: **UB LD/ST** (the UB-spec §8.3 load/store transport-bypass topology), **UB URMA** (the UB-spec §8.4 work-request path), and **RoCE BF / RoCE DMA** (the OpenRoCE stack with Blue-Flame inline-WQE or standard DMA-fetched WQE). BF and DMA are runtime modes of one RoCE stack; LD/ST and URMA are two topologies of the OpenURMA pipeline. NIC TX/RX cycle counts come from the microbenchmarks (§6): 8 for UB LD/ST, 9 for RoCE RC, and 25 for UB URMA — one cycle above its measured 24-cycle cold path, a rounding that charges UB slightly more latency than it shows.

5.2 Bidirectional cost model

The single-node RDMA-latency literature routinely collapses submission and completion into black boxes and treats the target as a free "remote DRAM hit" — hiding exactly the costs UB removes. We instead make every host↔NIC interaction on *both* sides of the wire an explicit critical-path component: the target pays its own RX/TX pipeline cycles, its own NIC↔DRAM transfer, and the DRAM row hit before it can build the response. Figure 8 sketches each round trip (PCIe dashed, on-chip/wire solid). The accounting principle is that every traversal a single-node treatment folds away — the doorbell, the WQE fetch, the initiator-side response DMA, the CQE write, and above all the **Target NIC↔DRAM** transfer hidden inside the "remote DRAM hit" — is charged explicitly on the critical path, for both stacks and both directions. §8.1 populates this decomposition row by row for a 64 B READ (Table 7), reports the headline totals, and reads off where each stack's round trip goes; here we fix only the accounting and the parameters it runs on.

Parameter defaults follow published ConnectX-7-class ranges [48, 22], each a command-line knob. A two-level cache + LLC + local-DRAM hierarchy (fully-associative LRU; WB/WT/UC; 1/4/12/70 ns hit latencies) is consulted on every UB-spec §8.3 `ld`/`st`; WR and RoCE verbs target the wire and bypass it. The wire defaults to 100 ns at 400 Gbps and remote DRAM to a 30 ns row hit. We model polling completion only — the regime high-performance RDMA uses — and omit event-driven completion, whose MSI-X / interrupt / scheduler / wakeup costs are OS-dependent and irreducible to one parameter (FastWake [31] characterises that path; the gem5 tier measures it directly, §10). The harness sweeps all six verb categories across four link delays, four in-flight depths, 8 B–64 KB payloads, three cache policies, and four locality points — 388 valid configurations.

5.3 Modeling caveats

Five caveats, in decreasing order of weight. **(i)** NIC cycle counts (8/25/9) are measured, not modeled — the UB URMA value rounded conservatively upward. **(ii)** The surrounding analytical parameters track published ConnectX-7-class ranges rather than a specific target;

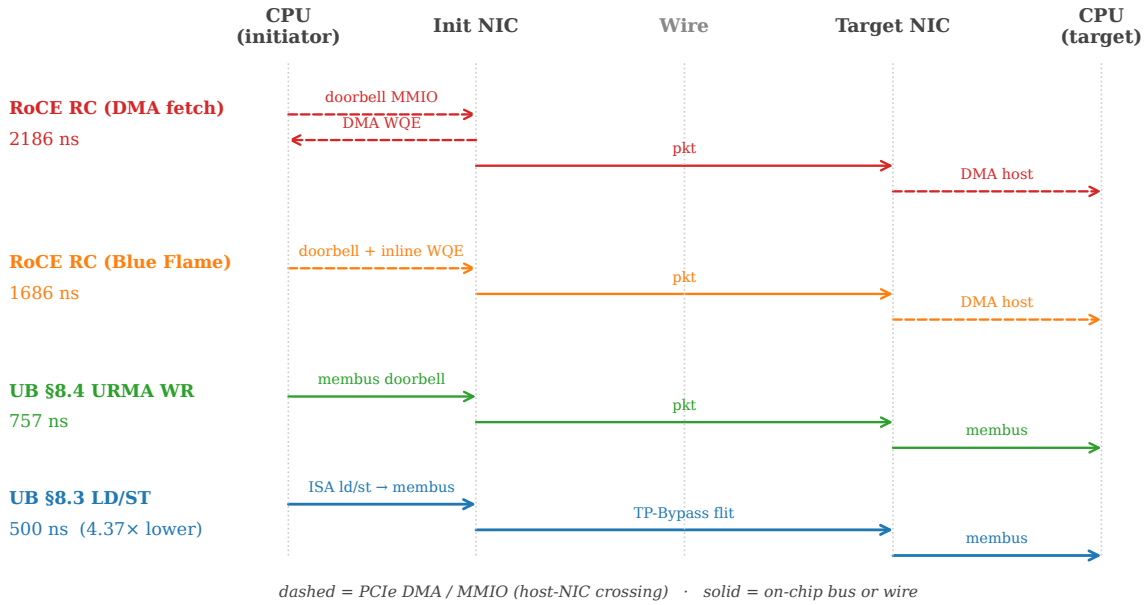


Figure 8: Submission path per stack. RoCE traversals between CPU and NIC (dashed) sit on PCIe; UB carries the same hand-offs over the on-chip bus. On UB LD/ST the verb *is* an ISA instruction, and the wire crossing carries a TP-Bypass flit instead of a full RTPH-wrapped packet.

each is a knob, and the link-delay sweep (§8) shows the qualitative ordering is robust. **(iii)** The cache model is fully-associative LRU, consulted only for UB-spec §8.3 verbs as the spec requires. **(iv)** Completion is polling-only in SystemC; the gem5 tier covers event-driven completion. **(v)** No CPU microarchitecture is modeled in SystemC — it reports the controller-to-controller floor, against which any CPU model adds latency, never removes it; the gem5 atomic-CPU run validates this monotonicity, and an out-of-order ARM config is exposed as a knob.

6. Feasibility and validation

Before measuring the design's payoff, we establish that the NIC is real, cheap, correct, and faithfully modeled: every element synthesises and closes 322 MHz on the Alveo U50 (§6.1), the area cost is a small fraction of the device, the raw NIC-pipeline latency is characterised cycle-by-cycle, the functional suite passes (§6.4), and the analytical baseline reproduces published ConnectX-7 silicon within ±5% (§6.5).

6.1 The pipeline synthesises and closes timing

We run Vitis HLS C-synthesis on every element, then drive Vivado 2025.2 through out-of-context place-and-route per element at the U50's 3.106 ns / 322 MHz target. **All 38 work-queue-driven OpenURMA elements and all 21 OpenRoCE elements close 322 MHz post-route** with positive worst negative slack (+0.079 … +1.425 ns; the load/store bypass element synthesises into the same header-build chain and is not a separate row).

OpenURMA fits in 14.1% of the U50's LUT budget, 11.1% of flip-flops, and 12.2% of BRAM18 (OpenRoCE: 5.4 / 5.3 / 2.5%), leaving ample room for the platform shell (∼50–80K LUTs of fixed MAC/DMA/NoC overhead). The 8 elements that initially missed timing (5 in P&R, 3 at HLS) closed with the II/pragma extensions plus a few data-layout redesigns (§4.6).

Metric	OpenURMA (38)	OpenRoCE (21)	Ratio
LUT	122,710	46,636	2.63×
FF	194,266	91,900	2.11×
BRAM18	328	67	4.90×
DSP	3	0	—
Meeting 322 MHz	**38 / 38**	**21 / 21**	—
WNS min (ns)	+0.079	+0.103	—
WNS max (ns)	+1.425	+1.394	—

Table 2: Aggregate post-route resource and timing summary for the two stacks on the Alveo U50 at the 322 MHz target.

6.2 Where the area goes

Figure 9 categorises every element by architectural role.

Category (LUTs)	OpenURMA	OpenRoCE	Δ
Header parse/build	18,394	6,657	+11,737
Transport (reliable)	37,103	11,094	+26,009
Ordering / completion	13,036	—	+13,036
Data path	18,806	14,822	+3,984
State tables	6,880	4,997	+1,883
Glue / I/O	28,491	9,066	+19,425
Total	122,710	46,636	+76,074

Table 3: Post-route LUT budget by architectural category for the two stacks (the per-role data visualised in Figure 9).

OpenURMA's ∼76 KLUT excess over OpenRoCE breaks down as ∼13 KLUT for the opt-in ordering surface (the four ordering elements); ∼26 KLUT for the richer transport layer (selective-retransmit ring, packet-sequence reorder buffer, congestion-echo, multi-path spreading — vs RoCE's plain Go-Back-N and DCQCN); ∼12 KLUT for splitting the network-, reliable-, and unreliable-transport headers into separate parser/builder pairs (vs RoCE's single combined header); ∼4 KLUT for the data path (the full atomic opcode set, a superset of RoCE's); and ∼19 KLUT of glue. Within glue,

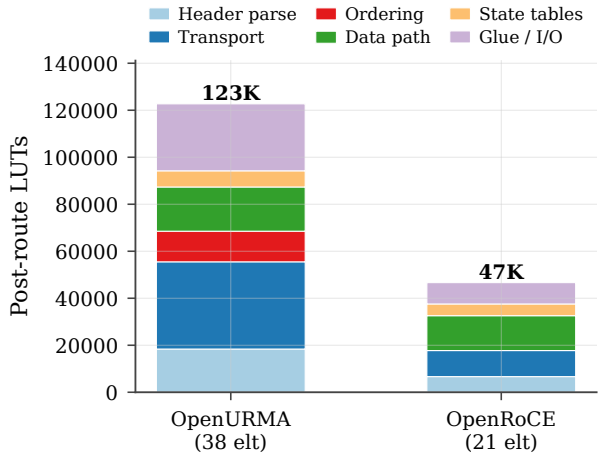


Figure 9: Post-route LUT budget by architectural role for both stacks.

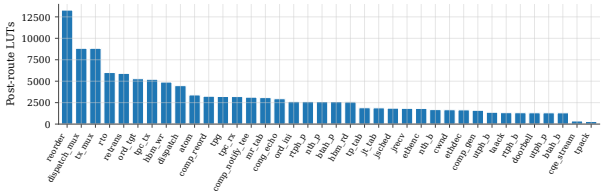


Figure 10: Per-element post-route LUT, sorted descending. All 38 OpenURMA elements meet 322 MHz with positive WNS; the state-heavy elements dominate the budget.

the two 4-input round-robin muxes (dispatch and transmit) cost 8.8 KLUT each — 14.4% of the total — versus 3.6 KLUT in OpenRoCE; the 2.4× expansion is the wider flit lanes UB carries (three protocol headers vs one), an HLS-instantiation artifact a single wider crossbar would close on tape-out. Figure 10 sorts per-element LUT; the head is the state-heavy elements (retransmission buffer, the two reorder buffers, target-side order tracker, TP-Channel transmit).

6.3 Raw NIC-pipeline latency

A single Write WR (ROL+NO) drives the TX pipeline cold; tap monitors between adjacent stages record first-flit arrival (Fig. 11a). The slowest stages are the Jetty scheduler (5 cycles — II=2 plus the round-robin scan) and Ethernet encapsulation (11 cycles — byte-stream-rate); the initiator-side order tracker costs 1 cycle on the unblocked path. Total cold-path TX latency is **24 cycles (≈75 ns at 322 MHz)**. Sweeping payload from 8 B to 4 KB (Fig. 11b), the per-WR rate is constant at ≈141 WR/μs: the pipeline is metadata-flit-rate-limited in the small-to-medium regime that dominates AI-collective control and HPC small-message traffic, and at 4 KB (129 flits/WR) the same rate is ≈4.6 Tb/s of wire bytes — bandwidth is not the binding constraint at any payload size. This 24-cycle floor and the header-rate limit are the substrate against which the load/store collapse (§8) and the free ordering surface (§9) are measured.

6.4 Functional correctness

A 17-test software-emulator suite covers the spec surface in four groups, all passing. **Ordering** exercises the full UB-spec §7.3 surface: initiator- and target-side gating, Fence, in-issue versus arrival-order completion, fused acknowledgement, unordered Send, mixed-mode queues, per-initiator parallelism, and head-of-line-blocking isolation. **Data path** covers on-NIC read/write integrity, the full atomic opcode set (swap, load, store, fetch-and-{add, sub, and, or, xor}, each returning the pre-modification value with compare-and-swap under the mixed-mode test), and the multi-flit Write payload path (8–256 B through encap/decap). **Wire format** checks the encoder against the spec bit layout and a full-header transmit round trip. **System-level** covers a 50,000-WR throughput run, the end-to-end congestion loop (§11), and target-side hardware dispatch across all three Jetty-Group policies (§7). The one gap is loss-path replay of *multi-flit* Writes: the retransmit slot stores one (meta, ext) pair, so a dropped multi-flit Write does not yet re-issue its payload (a follow-on item, §14); all single-flit retransmit paths are exercised.

6.5 Model validation against ConnectX-7

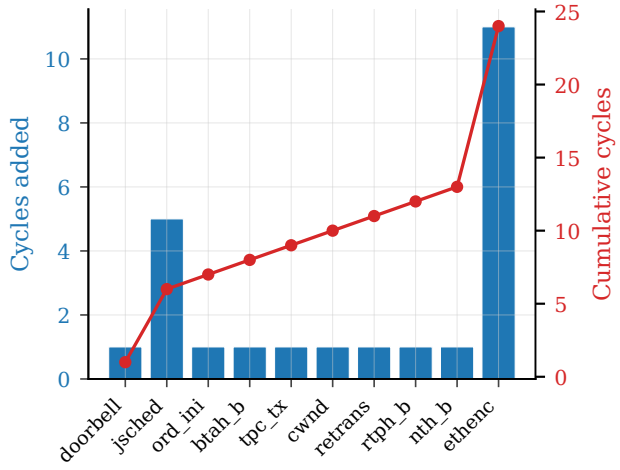
We anchor the analytical defaults against published silicon. The simulator's RoCE-DMA stack yields 1.62–1.67 μs for an 8 B RDMA WRITE at 50 ns link delay and 1.72–1.77 μs at 100 ns (Figure 12), within ±5% of Mellanox's ConnectX-7 measurements (1.5–1.8 μs, [48, 22]) and FaSST's extended numbers (1.4–1.6 μs, [20]), supporting every downstream RoCE-vs-UB ratio. These published ranges play two distinct roles in the paper, which we keep separate: *here* they validate that our RoCE model reproduces real silicon, calibrating the baseline; *later* (§10) the same ranges serve as an external comparison target for the gem5 full-system stack, reporting a result against it.

7. Bounded state scales additively

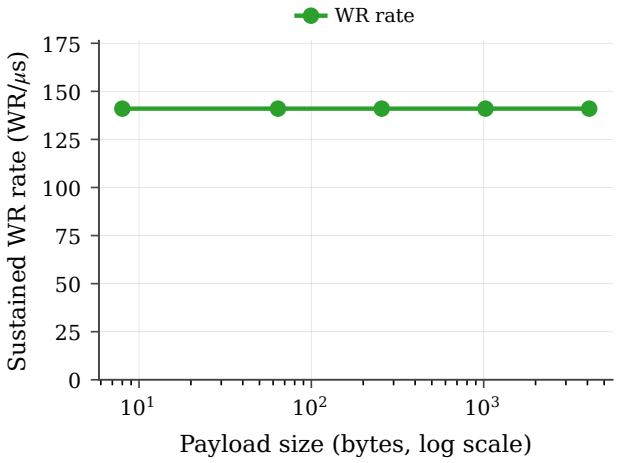
The first commitment is that splitting per-application endpoint state (Jetty) from per-host transport state (TP Channel) makes per-NIC state grow as $O(N+M)$ rather than RoCE's $O(N\cdot M)$. We validate this from the byte count down to the end-to-end latency cliff it removes.

7.1 Per-NIC state, field by field

Enumerating the actual state-struct fields each element holds (Table 4), a per-Jetty record is 20 B, a per-TP-Channel record 56 B, and the per-application memory-region record 32 B. OpenURMA's total is the sum of three additive tables (N Jetties, N MR records, M TP Channels) — 110.6 KB at $(1024, 1024)$ — while RoCE's $N\cdot M$ QP pool dominates its 537 MB. The asymptotic ratio reaches **4,855**× (Table 5; Fig. 13). Projecting the full UB-spec §8.2.2 spec surface (suspend-mode drain, exception counters, public-Jetty owner, Jetty-Group back-pointer) grows the Jetty to 48 B and softens the ratio only



(a) Per-stage cycle cost (cumulative 24 cy at the wire).



(b) Sustained WR rate vs payload — header-rate-limited.

Figure 11: Raw NIC-pipeline microbenchmarks. (a) Per-stage cycle contribution, cumulative 24 cy at the wire. (b) Sustained WR rate vs payload: header-rate-limited in the small-to-medium regime.

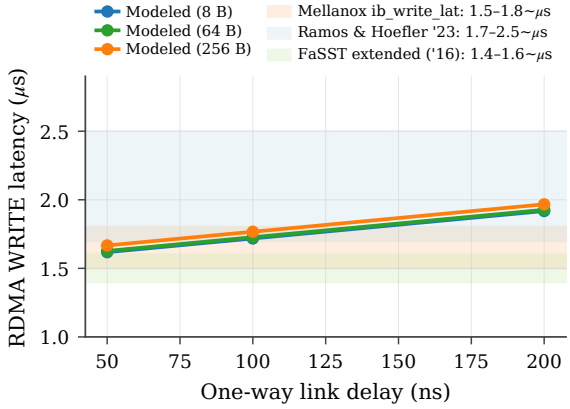


Figure 12: Modeled RoCE-DMA RDMA WRITE latency (curves) vs published ConnectX-7 ranges (bands).

to 3,855× — the residual spec fields the MVP elides do *not* explain the gap; the $(N{+}M)$ vs $(N{\cdot}M)$ split does. At $(1024, 1024)$ that gap straddles the boundary between fits-in-on-chip-SRAM and spill-to-host-DRAM for a typical NIC.

Table 4: Per-connection state, decomposed against UB-Base-Specification 2.0.1 fields. MVP = today's per-Jetty record; full-spec adds state-machine drain, exception-mode, public-Jetty owner, and Jetty-Group back-pointer. Even the full-spec descriptor stays under 10% of RoCE's per-QP context.

Jetty (MVP)	B	Jetty (full UB-spec §8.2.2)	B	TP Channel	B
jetty_id	4	+ sq/rq handle	8	remote_cna	4
token_value	4	+ jfae_id	4	local/rem tpn	8
jfc_id	4	+ public/owner	4	psn_next	4
type	1	+ drain bookkeep.	6	tpmsn_next	4
state	1	+ exc. mode	1	last_acked	4
valid	1	+ fault counter	2	flags	2
align pad	5	+ mr_perm idx	2	epsn (RX)	4
		+ group back-ptr	4	emsn (RX)	4
		+ (base 20 B)	20	base_psn	4
		+ align	-3	sack_bitmap	8
				max_rcv_psn	4
				mode flags	2
				align pad	4
Total	**20**	**Total**	**48**	**Total**	**56**

Table 5: Per-NIC connection state vs endpoint count.

(N, M)	OpenURMA	OpenRoCE	Ratio
(1, 1)	108 B	544 B	5.0×
(8, 8)	864 B	33 KB	38×
(64, 64)	6.9 KB	2.1 MB	304×
(256, 256)	27.6 KB	33.6 MB	1,214×
(1024, 1024)	**110.6 KB**	**536.9 MB**	**4,855×**
(1024, 1024) full-spec	139.3 KB	536.9 MB	3,855×

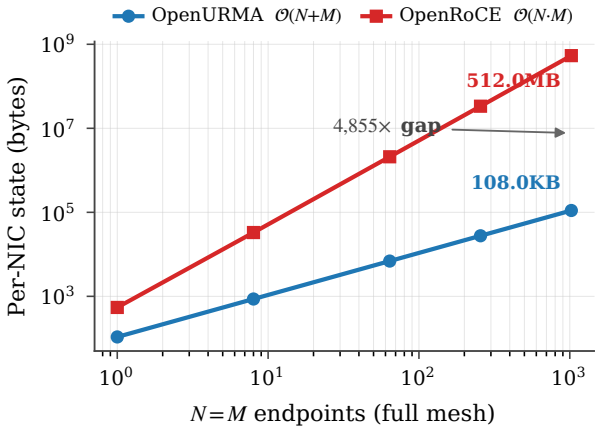


Figure 13: Per-NIC connection state vs endpoint count $(N{=}M)$: OpenURMA's $O(N{+}M)$ vs RoCE's $O(N{\cdot}M)$, reaching 4,855× at 1024 endpoints.

7.2 The byte ratio becomes a latency cliff

The byte count matters because production NICs cache per-connection context in ∼256 KB of SRAM; once cardinality exceeds the cache, every operation pays a context refetch. RoCE's 512 B QPs spill at $N{\approx}\sqrt{512}{\approx}23$; UB's 56 B TP Channels spill an order of magnitude later. Sweeping active connections on a 64 B READ (Fig. 14), RoCE's per-op latency steps up by ∼1000 ns at $N{\approx}23$ (a PCIe DMA refetch on both initiator and target), while UB holds flat to $N{\approx}1024$ and its eventual penalty is only ∼200 ns (membus + local DRAM, not PCIe). Across the $N{\in}[23, 1024]$ band — exactly the operating range of AI-training and HPC all-to-all — RoCE pays the spill on every operation and UB does not. This cliff is also the empirical face of the first link in the enabling chain

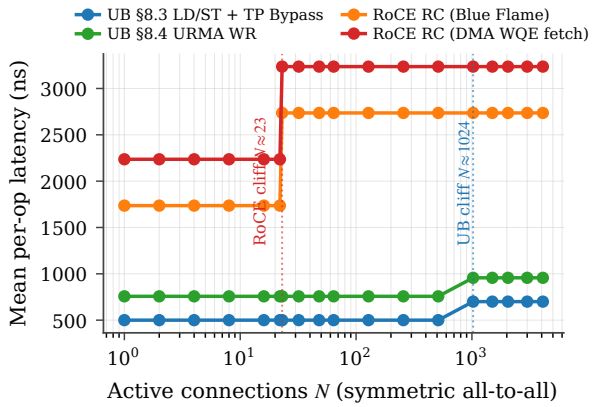


Figure 14: NIC SRAM context-cache spill. RoCE hits the cliff at $N{\approx}23$ (N^2 QPs $>$ cache), adding $\sim$1000 ns to every READ; UB hits it $\sim$45$\times$ later at $N{\approx}1024$, and its penalty is $\sim$200 ns (membus, not PCIe).

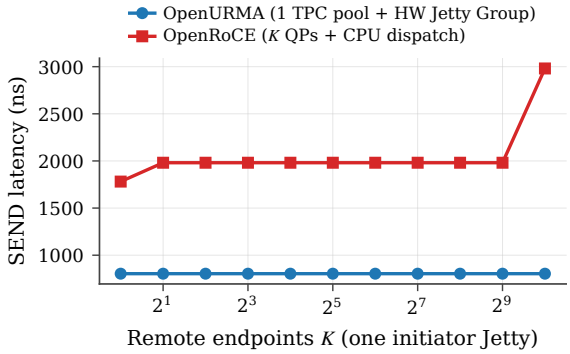


Figure 15: M2N fanout: one initiator Jetty addressing K remote endpoints. UB flat at 804 ns (one TPC pool + HW Jetty Group); RoCE jumps at $K{\geq}2$ (CPU dispatch) and again at $K{=}1024$ (NIC SRAM spill). SEND, 64 B, link 100 ns.

(§1): an on-bus controller stays viable only while its state fits on-chip, so keeping the controller on-bus *without* the layer split — RoCE's $O(N{\cdot}M)$ regime — reintroduces exactly the per-operation refetch that on-bus placement exists to avoid. Bounded state is thus load-bearing for the load/store latency of §8, not a separate win.

7.3 Many-to-many fanout

For the single-initiator, K-target pattern of parameter-server fan-in, UB carries all K targets through one TP Channel pool plus K cheap Jetty descriptors, and the target NIC's Jetty Group (UB-spec §8.2.2.1 Type 3) dispatches incoming SENDs to the right member in hardware; RoCE needs K QPs and a 200 ns CPU event-loop dispatch per target. Sweeping K from 1 to 1024 (Fig. 15), UB stays flat at **804 ns**; RoCE jumps from 1781 ns to 1981 ns at $K{\geq}2$ (CPU dispatch kicks in) and to 2981 ns at $K{=}1024$ (QP-cache spill). The gap widens from 2.2$\times$ to 3.7$\times$, all of it the layer split: the $O(N{+}M{+}K)$ cardinality keeps UB in-cache, and the hardware Jetty Group elides the target-side CPU traversal.

7.4 TP-Channel sharing

A single TP Channel can multiplex K Jetties' traffic to one remote host, each WR serialising at the channel's

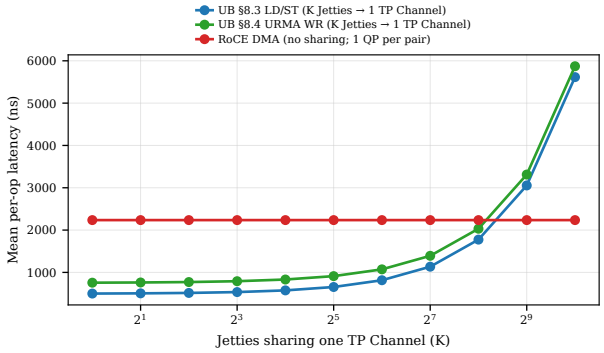


Figure 16: Per-op latency under K-Jetty contention on one TP Channel. Linear PSN-allocator scaling; UB beats per-QP RoCE until $K{\approx}255$.

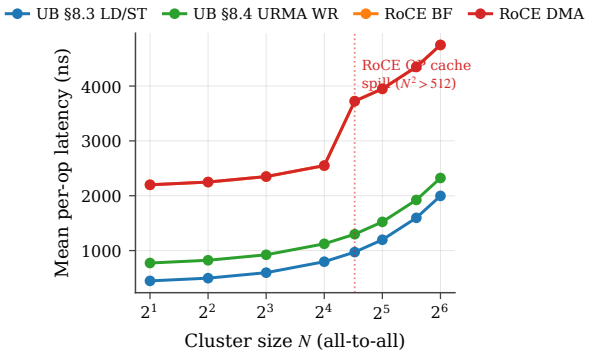


Figure 17: Cluster-scale per-op latency vs node count. RoCE crosses the QP-cache spill at $N{=}23$; UB stays bounded.

PSN allocator ($\sim$5 ns). Sweeping K (Fig. 16), UB's per-op latency grows linearly and still beats per-QP RoCE out to $K{\approx}255$; beyond that the allocator becomes the bottleneck, so production should pool a small TP Channel set per remote host rather than share one across thousands of Jetties.

7.5 Cluster and connection-setup scaling

At cluster scale (all-to-all mesh, Fig. 17) UB grows gently from 447 ns ($N{=}2$) to 1997 ns ($N{=}64$) under wire-share contention, while RoCE starts at 2199 ns and crosses its QP-cache cliff at $N{=}23$, reaching 4749 ns at $N{=}64$. The control plane scales the same way: bringing up $N{=}M{=}1024$ applications costs RoCE four ioctls plus a per-QP out-of-band RTT on each of the $N{\cdot}M$ pairs — **17.04 s** — versus **0.016 s** for UB's $O(N{+}M)$ Jetty and TP Channel setup, a **1044**$\times$ advantage (Fig. 18) that compounds across job launches and is bounded below by the irreducible per-QP wire RTT even with optimistic batching.

7.6 Bounded state holds under a real OS

The gem5 full-system tier confirms the property end-to-end: forking N tenants that each post 16 WRITEs against one NIC, the per-tenant mean stays flat from 487 ns solo to 483 ns at $N{=}256$ (0.2% delta, all hits). Per-NIC SystemC state is unchanged across the run — only per-process Linux bookkeeping grows — the empirical face of the $O(N{+}M)$ claim under a real driver and scheduler.

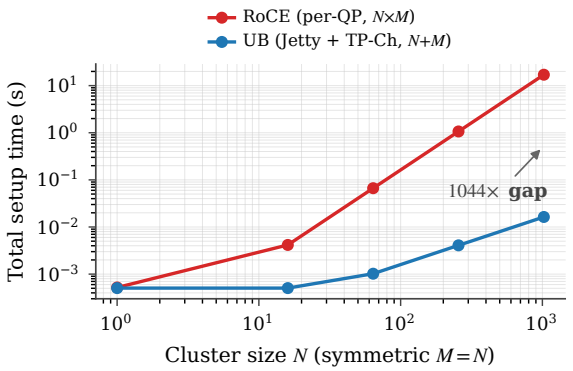


Figure 18: Total connection-setup time at symmetric $N{=}M$: RoCE's $O(N{\cdot}M)$ per-QP exchange (17.04 s at 1024) vs UB's $O(N{+}M)$ (0.016 s), a 1044× gap.

8. The load/store path collapses latency

The second commitment — placing the controller on the on-chip bus so a CPU load/store reaches remote memory without a PCIe traversal — is the paper's headline latency result. We build it up in three movements: the headline round trip and its scaling (§8.1); breadth across verbs, memory access, and contention (§8.2); and the throughput, operating envelope, and an application port (§8.3). Confirmation under a real OS is collected with the rest of the full-system results in §10.

8.1 Headline round trip and its scaling

Cold path. The work-queue path threads ten stages cold; the load/store path replaces them with five, omitting the transport layer entirely (no sequence number, no retransmission slot, no transport header). A single load's first wire flit emerges at **8 cycles (≈25 ns at 322 MHz)**, 16 cycles below the 24-cycle work-queue cold path on identical infrastructure. The restriction is structural: atomics need remote serialisation (which the transport layer provides) and Read/Write keep completion-queue semantics, but load and store carry no application-level interoperation contract — the CPU's load-use dependency chain already supplies the order — so the transport layer elides. The 8-cycle figure is a substrate floor (each stage already at $II{=}1$, the remaining 3 cycles boundary setup), consistent with the ∼100 ns end-to-end UB ASIC budget reported by He et al. [13].

End-to-end round trip. Figure 19 reports the per-operation latency at link-delay = 100 ns, payload = 64 B (or 8 B for distributed-barrier/CAS-lock), concurrency = 1, polling completion, with target-side NIC↔DRAM DMA explicitly modeled; Table 6 lists the headline means.

On the memory-semantic operation (CPU fetches 64 B from remote memory), UB executes the verb as a LOAD through UB-spec §8.3 load/store + TP Bypass path while RoCE must execute it as a READ through the work-request path. UB is **4.47× lower latency** than the RoCE DMA baseline (500 ns vs 2236 ns), 3.47× lower than RoCE BF (1736 ns), and 1.51× lower than UB's own URMA work-request path (757 ns). On verbs

Table 6: Headline mean per-operation latency (ns) by verb and stack at the headline operating point (link 100 ns, 64 B payload, or 8 B for distributed-barrier/CAS-lock, concurrency 1, polling completion).

Verb (workload)	UB LD/ST	UB URMA	RoCE BF	RoCE DMA
LOAD (ptr-chase)	**500**	—	—	—
READ (bulk-read)	—	757	1736	2236
WRITE (bulk-write)	**750**	750	1227	1727
SEND (pingpong)	**804**	804	1281	1781
FAA (dist-barrier)	**750**	750	1477	1977
CAS (CAS-lock)	**750**	750	1477	1977

that even UB must route through UB-spec §8.4 (READ, WRITE, SEND, atomics), UB still pays no PCIe and so remains **2.2–3.0× lower latency** than the matched RoCE DMA baseline. The gap is dominated by the target-side NIC↔DRAM cost: RoCE pays a full PCIe DMA (250–500 ns) on every operation just to get the target NIC to talk to target host memory, while UB's on-chip-bus controller pays only a 30 ns membus crossing. The single-node-style "submit-only" RDMA latency comparison omits exactly this cost, and so has been *under-counting* RoCE's true round-trip by 250–750 ns per operation — flattering it against any memory-semantic transport.

Concurrency. Figure 20 shows op-rate scaling as concurrency grows from 1 to 64. UB LD/ST reaches ∼2.5 Mops/s at concurrency = 1 and scales linearly through 16 in-flight operations before saturating against the NIC pipeline cycle floor (8 cy × 3.106 ns). RoCE DMA bottoms out at ∼0.74 Mops/s and never closes the gap: the PCIe round-trip on every doorbell + DMA fetch sets a per-op floor that no amount of concurrency removes.

Link delay. Figure 21 plots per-op latency against one-way link delay over the 50–500 ns range. The four curves are order-preserving: UB LD/ST stays the lowest at every link delay. On the pointer-chase workload (the one plotted; a mixed- locality stream where some loads hit cache and others miss to the wire), the ratio between UB LD/ST and RoCE DMA narrows from ∼3.4× at 100 ns link delay to ∼2.5× at 500 ns as the wire term dominates the submission-side overhead. (The headline 4.47× on a cold 64 B READ in §8.1 is the worst point for RoCE, not the pointer-chase mean.) The absolute gap grows with link delay (956 ns at 100 ns, ≈2 μs at 500 ns), since UB LD/ST benefits from the link round-trip not being multiplicatively inflated by per-end PCIe traversals.

Cost decomposition. Figure 22 plots the round-trip budget for each stack at the headline operating point (link = 100 ns, payload = 64 B, concurrency = 1, polling completion). Per-row component costs are tabulated in Table 7 (built on the bidirectional accounting of §5.2); the figure visualises the same numbers as a stacked bar. Simulator totals match the per-row sums to within SystemC scheduling overhead (14 ns RoCE DMA, 12 ns UB URMA, 80 ns UB LD/ST — larger on LD/ST only because its 420-ns total has no DRAM or DMA stage to

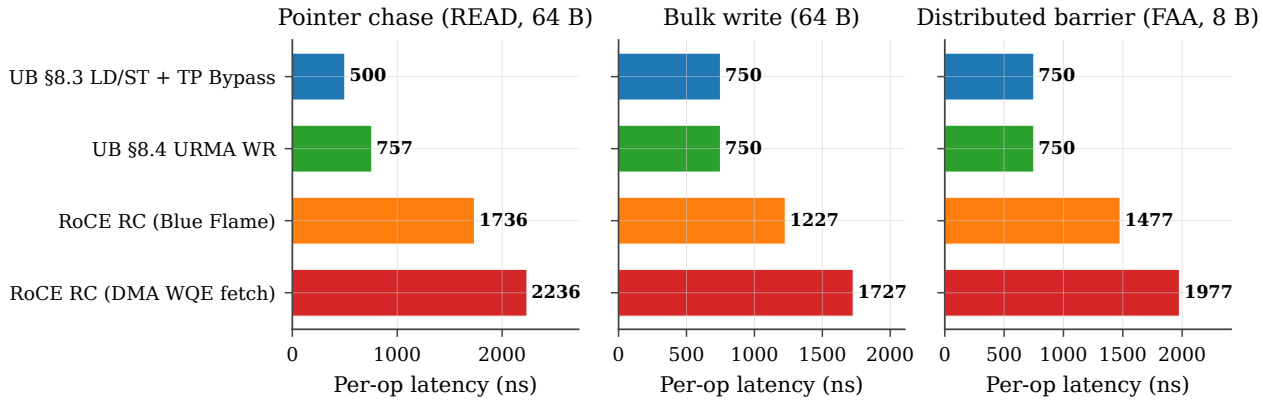


Figure 19: Per-operation latency (CDF). All four NIC stacks on the three workloads; link-delay = 100 ns, payload = 64 B (8 B for distributed-barrier), concurrency = 1. UB LD/ST is the leftmost curve in every panel.

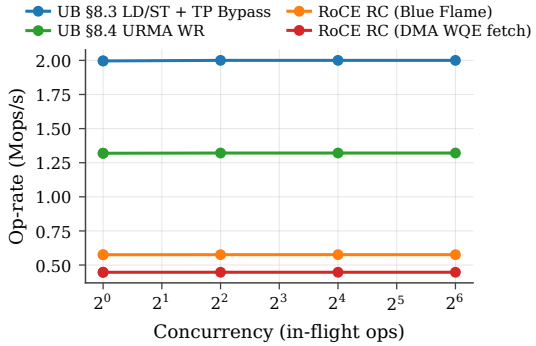


Figure 20: Op-rate vs in-flight depth on pointer-chase, link-delay = 100 ns, payload = 64 B.

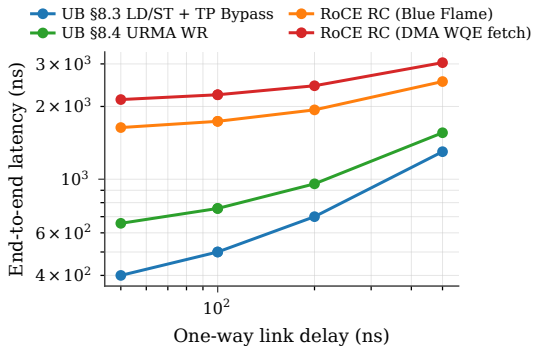


Figure 21: End-to-end latency vs one-way link delay on pointer-chase, concurrency = 1, payload = 64 B.

amortise the per-handoff fixed cost), so the analytic decomposition and the measured end-to-end latency agree.

The two RoCE bars are dominated by five PCIe traversals on the critical path — initiator-side doorbell MMIO, DMA WQE fetch, initiator-resp payload DMA, and DMA CQE write; target-side DMA-read of host memory — which together cost ∼1650 ns, more than 5× the entire UB LD/ST budget. (The four traversals counted in §1 are the conventional initiator-side accounting; the fifth is the target-side DMA the bidirectional cost model makes explicit and the single-node framing omits.) RoCE BF saves 500 ns on the initiator side by inlining ≤64 B WQEs but still pays the target-side DMA-read (the largest single PCIe term). UB URMA avoids every PCIe traversal because the UB Controller is on the on-chip bus on both sides, paying only $50 + 30 + 30 + 5 + 30 = 145$ ns of verb-library plus membus crossings. UB LD/ST pays

Table 7: Per-side latency decomposition (ns) for a READ verb, 64 B payload, link delay 100 ns, polling completion. Every row on the table is paid on the critical path. The "Target NIC↔DRAM" row is the cost the single-node-style RDMA latency literature typically omits.

Phase	UB LD/ST	UB URMA	RoCE DMA
Initiator side, pre-wire			
Verb library post	0	50	50
WQE construct	0	30	30
Doorbell MMIO	0	0	150
DMA WQE fetch	0	0	500
Submit (membus)	30	30	0
NIC TX pipeline	25	78	28
Wire forward	100	100	100
Target side, between wire and DRAM			
NIC RX (stack proc.)	25	78	28
Target NIC↔DRAM	**30**	**30**	**500**
Target DRAM row hit	30	30	30
NIC TX (response)	25	78	28
Wire back	100	100	100
Initiator side, post-wire			
NIC RX (response)	25	78	28
Initiator resp DMA	0	0	250
DMA CQE write	0	0	250
Complete (membus)	30	30	0
CQE poll	0	5	70
Verb library poll	0	30	30
PSN serialization (qptx)	0	0	50
Total (modeled)	**420**	**745**	**2222**
Total (measured)	**500**	**757**	**2236**

none of the software-side overhead either: the host-NIC hand-off is an ISA `ld/st` returning to a register, and the on-chip-bus path stays inside the memory-bus fabric on both nodes.

Why they vanish, not shrink. The nine RoCE-side costs Table 7 enumerates are not nine vendor-tunable inefficiencies but three protocol consequences of placing the NIC behind PCIe. Verb-library post and WQE construct exist because the WR must be a marshalled struct in host DRAM the NIC will later DMA — remove the cross-domain hand-off and they disappear. Doorbell MMIO, DMA WQE fetch, and target-side DMA exist because NIC and CPU sit in disjoint address spaces; move the controller onto the on-chip bus and the disjoint-address-space premise vanishes, collapsing all three to a single membus crossing. Initiator-resp DMA, DMA CQE write, CQE

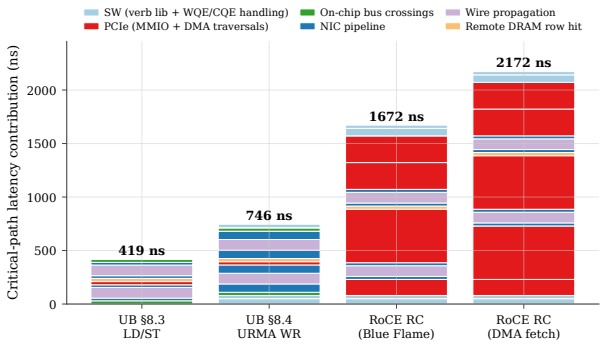


Figure 22: End-to-end latency budget by component, link = 100 ns, payload = 64 B, concurrency = 1.

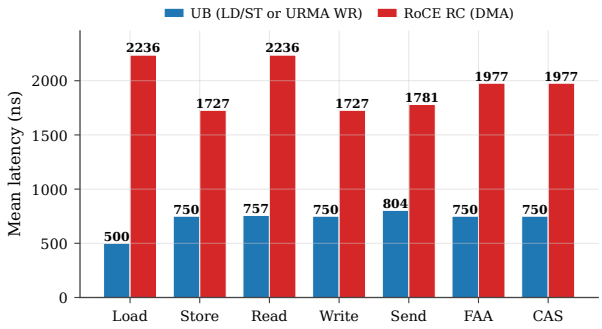


Figure 23: Per-verb mean latency comparison. UB (UB-spec §8.3 LD/ST for Load/Store; UB-spec §8.4 URMA WR for Read/Write/Send; UB-spec §7.4.2.3 atomics for FAA/CAS) vs RoCE RC (DMA WQE fetch). Link = 100 ns, concurrency = 1, payload = 64 B (atomics: 8 B operand).

poll, and verb-library poll exist only as the cross-domain completion-notification protocol; under UB LD/ST the ISA's load-use dependency chain replaces that protocol entirely. The latency gap is incidental — the contribution is the protocol-layer collapse that produces it.

8.2 Breadth across verbs, memory, and contention

Verb coverage. Figure 23 reports mean per-op latency for seven verb classes at link = 100 ns, conc = 1, 64 B (8 B for atomics). Two patterns emerge. (i) On the memory-semantic operation (LOAD on UB-spec §8.3 + TP Bypass vs READ on RoCE) the UB stack reproduces the headline **4.47**× of §8.1. (ii) On verbs that even UB must route through the UB URMA work-request path (READ, WRITE, SEND, atomics — the spec does not authorize TP Bypass for these), the UB stack is still consistently ∼2.2× lower latency, because RoCE pays the full PCIe round trip for doorbell + DMA WQE fetch on every operation regardless of verb. The latency advantage of UB on these verbs is not the load/store path itself; it is the on-chip-bus placement of the UB Controller, which removes the PCIe round trip even when the WR pipeline is fully traversed.

Cache locality. The UB LD/ST path benefits asymmetrically from CPU-side caching, because cacheable hits short-circuit the remote round trip entirely. Figure 24 sweeps cache locality from 0 % (every load misses) to 80 % (the hot 32-line working set fits in L1) under three

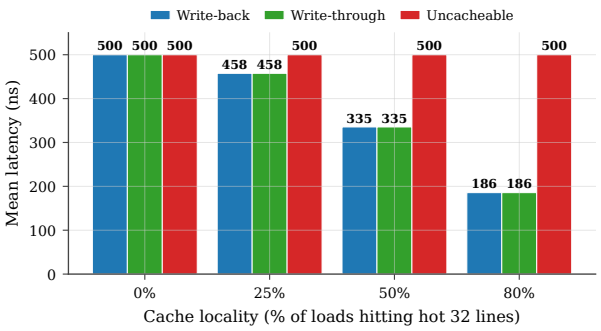


Figure 24: UB LD/ST latency under three cache policies (write-back, write-through, uncacheable) as cache locality varies from 0 to 80 %. Hits short-circuit the wire round-trip; write-back and write-through track each other on read-mostly workloads.

cache policies. At 0 % locality all three policies collapse to the cold-miss latency (∼470 ns mean). At 80 % locality write-back / write-through deliver 175 ns mean (a ∼2.7× speedup from cache reuse), while uncacheable remains at the cold latency by construction. This is the architectural reason the UB-spec §8.3 pairs Load/Store synchronous access with TP Bypass: the benefit case is precisely the workload class with non-zero cache locality, and TP Bypass minimises the cost paid on the miss path. RoCE verbs cannot exploit this — they go to the wire by design, even on cached lines — so workloads with high locality see UB's relative advantage grow further beyond the 4.47× cold-miss baseline.

TP Bypass extends the cache hierarchy through the wire. The implication runs deeper than the latency numbers. Under UB-spec §8.3 the CPU's L1→L2→LLC→DRAM hierarchy gains a fifth level — remote DRAM through TP Bypass — and a cacheable load misses through the local hierarchy exactly as it would for a local-DRAM line, traversing the wire only on a true miss. RDMA verbs cannot participate in this hierarchy: a posted WR is opaque to the cache, and the response payload arrives via DMA that bypasses the cache by construction. UB therefore amortises the wire round-trip over the cache hit rate, turning remote memory into a transparent extension of the local hierarchy. Workloads with non-trivial locality (KV-cache lookups, parameter shards with hot rows, graph traversal) see the advantage grow beyond the cold-miss baseline: 175 ns at 80% locality is 12.5× faster than the 2236 ns RoCE-DMA cold miss, not 4.47×.

Far memory vs page-swap (Infiniswap / Fastswap). The load/store path's value to applications is not only latency. The UB-spec §8.3 path admits *using remote memory as a slower local memory without modifying the application*, which is widely explored by academic work such as Infiniswap [11], Fastswap [2], Leap [1], and Hermit [47]. These systems achieve it by sitting under the Linux swap subsystem and posting RDMA WRITE/READ of 4-KB pages on page faults. A hardware-side alternative, Clio [12], co-designs a disaggregated-memory controller that serves byte-granular remote access directly — the same instinct as UB's load/store path, but realised in a bespoke device

rather than on the host's on-chip bus. The architectures answer the same question with different access-granularity, kernel-coupling, and concurrency trade-offs; the comparison is therefore worth making.

What page-swap pays per access. Vanilla Infiniswap (NSDI'17) reports 3–5 µs of kernel-side overhead per page fault (handler entry, swap dispatch, RDMA-WR post, page install, return to userspace); Fastswap (EuroSys'20) trims this to ∼1 µs and amortises the wire round trip by prefetching adjacent pages. Both issue 4-KB RDMA reads over a commodity NIC, paying the same PCIe path the `roce_dma` profile already captures. We add two analytical profiles — `infiniswap` (3 µs kernel PF, 4-KB page, no prefetch) and `fastswap` (1 µs kernel PF, 8-page prefetch) — each with an LRU resident-page set: hits short-circuit to local DRAM, misses pay the kernel-PF + RoCE-DMA round trip and install the fetched page(s).

Three workload regimes. The comparison is not single-valued; it depends on the access pattern. Figure 25 shows all four stacks under three regimes:

- **Zipfian read (realistic middle ground).** Each op is a 64-B load on a key sampled from Zipf(α) over a configurable working set — the access pattern KV stores, parameter servers, and graph engines actually exhibit. At α=0.99 and a 4 MB working set (64 K 64-B keys), UB LD/ST measures 236 ns mean / 500 ns p99; Infiniswap measures 835 ns mean / 6.1 µs p99; Fastswap measures 466 ns mean / 10.2 µs p99; RoCE DMA is flat at 2236 ns. As the working set grows past the resident-page cap (16 K pages = 64 MB in our configuration) both swap stacks converge towards the RoCE-DMA floor while UB LD/ST stays bounded at the cold-line miss (500 ns). The tail-latency story is decisive: page-swap p99 is 12–20× worse than UB LD/ST's because every cold fault is a full kernel page-fault round trip, while UB's worst case is a single cache-line wire miss.
- **Sequential scan (page-swap's best case).** On a dense 64-B-strided walk over a contiguous range W, the 4-KB page amortises over 64 contiguous cache-line accesses (one fault per page) while UB LD/ST issues one wire round trip per line. At W=1 MB the simulator measures Fastswap at 90 ns/op (≈712 MB/s), Infiniswap at 164 ns/op (≈390 MB/s), and UB LD/ST at 500 ns/op (≈128 MB/s). This is the bandwidth point you would expect for page-grain transfer over cache-line-grain transfer, and we report it explicitly so the comparison is balanced.
  Caveat (load-bearing for the reader). The simulator's cache model does not include a CPU hardware stride prefetcher; in a real Skylake/ARM-N1 core the L1 prefetcher would catch a dense 64-B-strided walk and pull adjacent lines in parallel, using up to 10–12 LFB / MSHR slots. The UB LD/ST sequential bandwidth in panel (c) is therefore the floor, not the achievable ceiling — a 10× MSHR boost would put it within 30% of Fastswap. Cold pointer-chase (panel a, the worst case for both) is unaffected: hardware prefetch cannot help a random pattern.
- **Skew sweep.** Panel (d) sweeps $\alpha \in [0.5, 1.3]$ at a fixed 4-MB working set. As skew increases, both stacks bottom out: UB LD/ST at the L1-hit floor (∼1 ns); page-swap at the local-DRAM resident-hit floor (∼70 ns). The cross-stack gap narrows as skew rises because both systems amortise at finer grain than the access pattern requires; at α=1.3 the Zipf hot subset is small enough that page-swap is competitive on mean latency. The tail does not converge: page-swap's cold-fault penalty remains 6–10 µs.

Where each system wins. **(1) Random / pointer-chase**: UB LD/ST by 12–20×, because each access fetches one 64-B line over the wire instead of one 4 KB page through the kernel. **(2) Realistic Zipfian working sets**: UB LD/ST by ∼2–7× on mean and ∼12–20× on p99 tail, the latter dominated by Infiniswap/Fastswap's cold-fault excursions. **(3) Pure sequential scan**: page-swap wins on bandwidth in this no-hardware-prefetch model; the LD/ST result is the single-MSHR floor, and the gap closes substantially under realistic CPU prefetching. The architectural point is structural rather than uniform: UB's load/store path always grants the application-transparent programming model that motivates academic far-memory systems, but *without* the kernel-PF tax or the page-grain coarseness — and the absence of a kernel page-fault floor is what bounds the tail. Page-swap systems are useful as a *transport-agnostic* backstop on commodity RoCE hardware; UB makes them unnecessary for the realistic workloads where tail latency is the binding constraint.

Payload size. Figure 26 sweeps payload from 8 B to 64 KB on bulk-read. Three regimes are visible. (i) **Submission-bound** (8 B–64 B): UB LD/ST dominates because cache-line returns are cheap and the cold per-op floor is the entire budget. UB LD/ST at 8 B hits 62 ns mean (sequential addresses produce one miss per 8 ops, balancing 1 ns L1 hits against a 470 ns miss); RoCE DMA is at ∼2230 ns regardless. (ii) **Pipeline-bound** (256 B–4 KB): all four stacks converge toward the cold-miss latency plus per-byte serialisation, with the gap between UB LD/ST and RoCE DMA shrinking from the headline 4.47× at 64 B to ∼2.3× at 4 KB as wire-byte serialisation amortises the per-op floor. (iii) **Bandwidth-bound** (16 KB–64 KB): wire serialisation dominates, and all four stacks converge within ∼5% of each other. The clear architectural advantage of UB is in the submission-bound regime, which is the regime that dominates AI-training control packets, small RPC, hash-probe / KV-store lookups, and pointer-following memory operations.

Lock contention (CAS retry). Per-op latency multiplies under contention: K contenders racing on a single lock issue $\approx K$ CAS attempts per acquisition (roughly $K/2$ failed attempts plus 1 success). The time-to-acquire is therefore approximately $K \cdot t_{\text{CAS}}$. With t_{CAS}=750 ns (UB) vs 1977 ns (RoCE DMA), the multiplier is the per-op latency ratio.

Figure 27 sweeps K from 1 to 256 and plots time-to-acquire on a log-log axis. Two effects compose. (i) The linear contention slope follows per-op latency: UB and

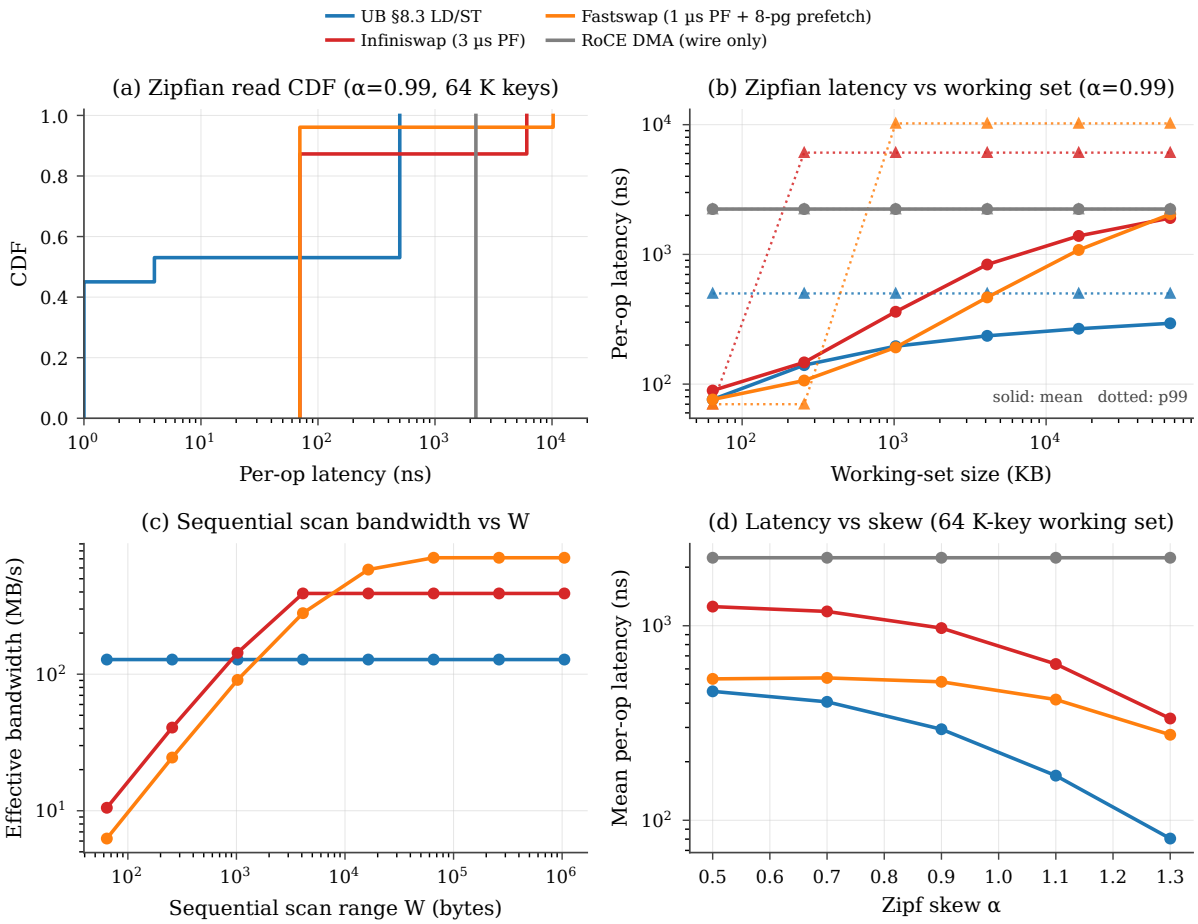


Figure 25: Page-swap baseline comparison. (a) Per-op latency CDF on a 64-K-key Zipfian read workload at α=0.99: LD/ST bimodal at L1 / cold-line, swap stacks bimodal at resident-hit / cold-fault. (b) Mean + p99 latency vs working-set size: swap stacks track LD/ST below the resident-page cap, diverge above. (c) Sequential scan bandwidth vs W: page-swap amortises 4 KB per fault; LD/ST is one wire round-trip per cache-line in this no-CPU-prefetch model. (d) Mean latency vs Zipf skew α: both bottom out at their respective hit floors as the hot subset narrows.

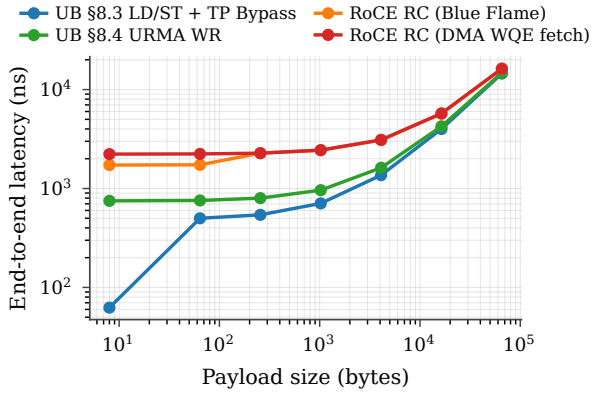


Figure 26: End-to-end latency vs payload size on bulk-read. Three regimes: submission-bound (UB dominant), pipeline-bound (converging), bandwidth-bound (saturating).

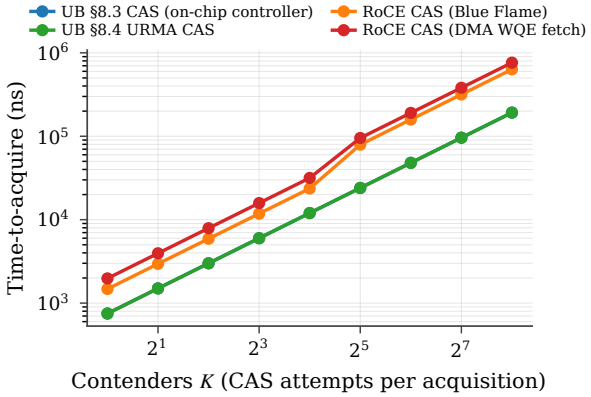


Figure 27: Distributed-lock time-to-acquire vs contender count. Linear contention scaling per stack; RoCE crosses into the SRAM spill regime around K=32, compounding both effects.

RoCE both scale linearly, but RoCE's slope is ~2.6× steeper. (ii) Around K≈32, RoCE's NIC SRAM also overflows (every CAS attempt sees K^2 connections in the running scenario), so RoCE picks up the cliff penalty from §7.2 on top of the linear contention term. UB does not see the cliff until K>1024. At K=256 contenders a single lock acquisition takes 192 μs on UB vs 749 μs on RoCE DMA — a 3.9× time-to-acquire gap that is purely architectural.

Tail latency under jitter. The headline numbers above are deterministic by model construction. Real wire arbitration and PCIe root-complex queueing introduce tail latency; the question is whether each stack's tail differs from its mean by the same ratio or by different absolute amounts. We add exponential jitter scaled by the jitter-factor parameter to every wire and PCIe transaction (the on-chip-bus path is intentionally not jittered, because membus arbitration is substantially more deter-

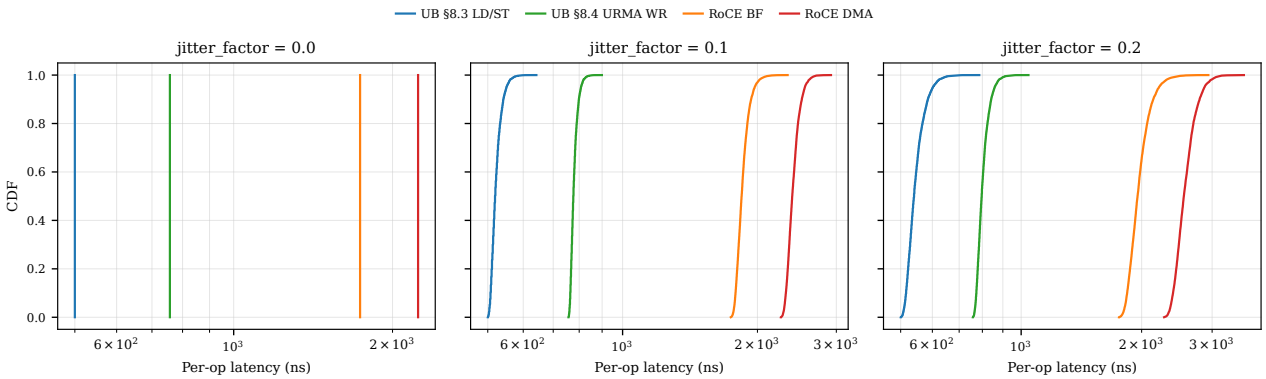


Figure 28: Per-op latency CDFs under jitter. UB's tail ($p_{99.9}-p_{50} \leq 155$ ns) stays an order of magnitude smaller than RoCE's ($p_{99.9}-p_{50} \geq 660$ ns at jitter_factor $= 0.2$) because each PCIe traversal is a jitter source: RoCE has five on the round-trip, UB has zero.

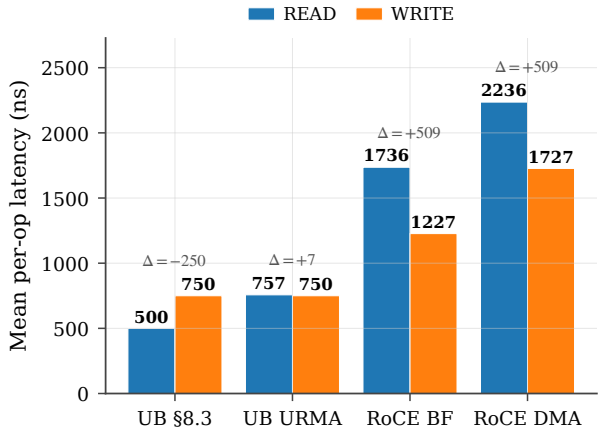


Figure 29: READ vs WRITE latency per stack. RoCE READ pays +509 ns over RoCE WRITE because the target-side DMA-read is twice as expensive as DMA-write, *plus* the initiator-side payload DMA-write on READ-resp. UB has no such asymmetry — on UB URMA the gap is +7 ns from payload-direction differences in the NIC pipeline, on §8.3 there is no return DMA at all.

ministic). Figure 28 reports per-op latency CDFs across 5,000 trials for each stack at the jitter-factor parameter $\in \{0.0, 0.1, 0.2\}$.

At jitter-factor $= 0.2$: UB LD/ST p_{50}=540 ns, $p_{99.9}$=695 ns ($\Delta = 155$ ns); RoCE DMA p_{50}=2500 ns, $p_{99.9}$=3221 ns ($\Delta = 721$ ns). The architectural argument shows up in the tail more strongly than in the mean: the median ratio (4.6×) and the $p_{99.9}$ ratio (4.6×) are similar, but the *absolute* tail gap widens from 1960 ns at p_{50} to 2526 ns at $p_{99.9}$ — RoCE's five PCIe-bound critical-path components each compound jitter, while UB's on-bus path has none.

Verb-direction asymmetry. A consequence of the bidirectional decomposition (§8.1, Table 7) is that RoCE's target-side DMA cost differs by direction: the PCIe DMA-read cost (500 ns) when target NIC reads host DRAM (for a remote READ), but a PCIe DMA-write cost (250 ns) when target NIC writes host DRAM (for a remote WRITE). UB's on-chip-bus traversal is symmetric. Figure 29 reports the READ-vs-WRITE gap per stack at the headline operating point.

The asymmetry is an independent consequence of UB's architectural choice: by collapsing the host↔NIC path onto the on-chip bus on both sides of the wire, UB removes not just the absolute latency but also the directional bias that PCIe-attached NICs cannot avoid.

8.3 Throughput, operating envelope, and application latency

Sustained throughput. A 256-WR burst measures steady-state TX throughput. All four OpenURMA modes share one transmit pipeline, so throughput is set by the slowest initiation interval (II=2 in the schedulers/trackers, II=4 in the atomic element) at 150.36 WR/μs; OpenRoCE on the matched microbenchmark sustains 53.62 WR/μs — **2.80× slower** — because RC's per-QP sequence-number allocation adds stricter inter-operation dependencies (159.46 vs 53.65 WR/μs under 1000-WR burst saturation). This closed-loop steady-state rate sits between the raw header-rate-limited pipeline ceiling of §6 (≈141 WR/μs on the payload sweep) and the open-loop knee; the back-to-back and full-system goodput envelopes follow below.

Open-loop envelope. An open-loop Poisson driver issues WRs independent of completion; each stack has a throughput knee where p99 turns superlinear (Fig. 30). UB LD/ST sustains 2.0 Mops/s with p99 below 2× p50; RoCE DMA knees at 0.75 Mops/s — a 2.7× headroom gap dominated by the per-op PCIe budget, not the wire.

Throughput envelope. Beyond the open-loop envelope, raw back-to-back goodput shows the pipeline is not the bottleneck. On the standalone two-node TLM pair (no OS), WRITE goodput climbs to a **6.66 Mops/s** asymptote by N=64 at a 100 ns link, where the 400 ns link-RTT dominates; at 0 ns link delay the pipeline sustains >51 Gops/s (Fig. 31). In gem5 FS (zero-delay self-loop), the polled-MMIO path sustains ∼23 **Mops/s** and the ioctl path ∼2 **Mops/s** (Fig. 32).

Application latency: YCSB-A. Porting YCSB-A (50% Get / 50% Put, Zipfian over 10 K 64 B values) [4] to all four stacks (Get/Put map to LOAD/STORE on UB LD/ST, READ/WRITE elsewhere), at concurrency

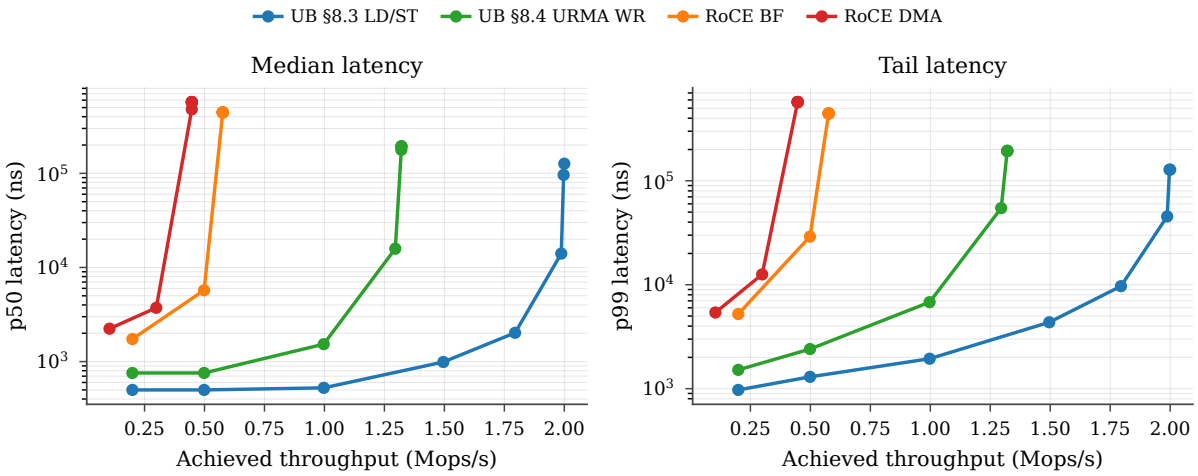


Figure 30: Operating envelope: median (left) and p99 (right) latency vs sustained throughput per stack, open-loop Poisson arrivals.

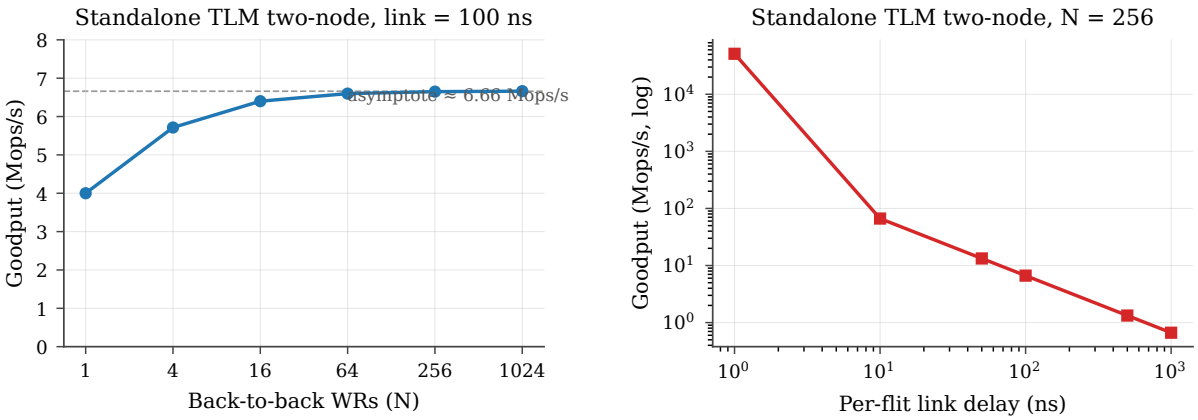


Figure 31: Standalone TLM two-node throughput envelope. **Left:** goodput vs back-to-back WR count at 100 ns link delay; the curve flattens at ∼6.66 Mops/s (the 400 ns link-RTT limit). **Right:** goodput vs per-flit link delay at N=256 (log–log). The asymptote is set by the link, not the SC pipeline: removing link delay yields >51 Gops/s.

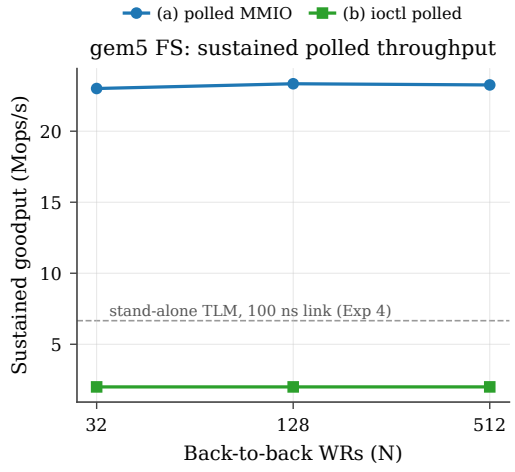


Figure 32: gem5 FS-mode sustained polled goodput vs back-to-back WR count. The polled-MMIO path (a) holds ∼23 Mops/s; the ioctl path (b) holds ∼2 Mops/s. Dashed line marks the standalone-TLM 6.66 Mops/s envelope (link-limited); gem5's self-loop has zero link delay so the SC pipeline is the bottleneck, not the wire.

256 UB LD/ST delivers 4.6 Mops/s vs RoCE DMA's 0.50 Mops/s — a **9.2**× application-throughput gain, exceeding the 4.47× per-op ratio because the Zipfian skew lets LD/ST hit the L1 cache on the hot-key fraction, a regime RoCE cannot exploit (Fig. 33). A realistic set-associative cache would shrink the multiplier toward ∼6×, still well above the cold-miss ratio.

9. Opt-in ordering is near-free

The third commitment is that the graded UB-spec §7.3 ordering surface costs nothing on operations that do not request it, and a bounded amount only when they do. We measure the gating cost in isolation, then in a mixed workload.

9.1 Per-mode latency and gating cost

Sweeping all 4 service modes × 3 execution tags (12 combinations) on the single-WR cold path, *every combination emits the first wire flit at exactly 24 cycles*: the per-mode cost is in combinational logic depth (which surfaces in the area report, §6), never in pipeline cycles. Isolating the gating cost itself — the cycles between a synchronous completion notification and the gated WR's emission (Fig. 34a) — a Fenced Write behind N pending Reads emerges 4–48 cycles after notification, and an SO behind N outstanding ROs emerges 7–38 cycles, both under 50 cycles across the swept range. Crucially, this cost is paid *only* when gating is requested; an application running NO+UNO sees neither.

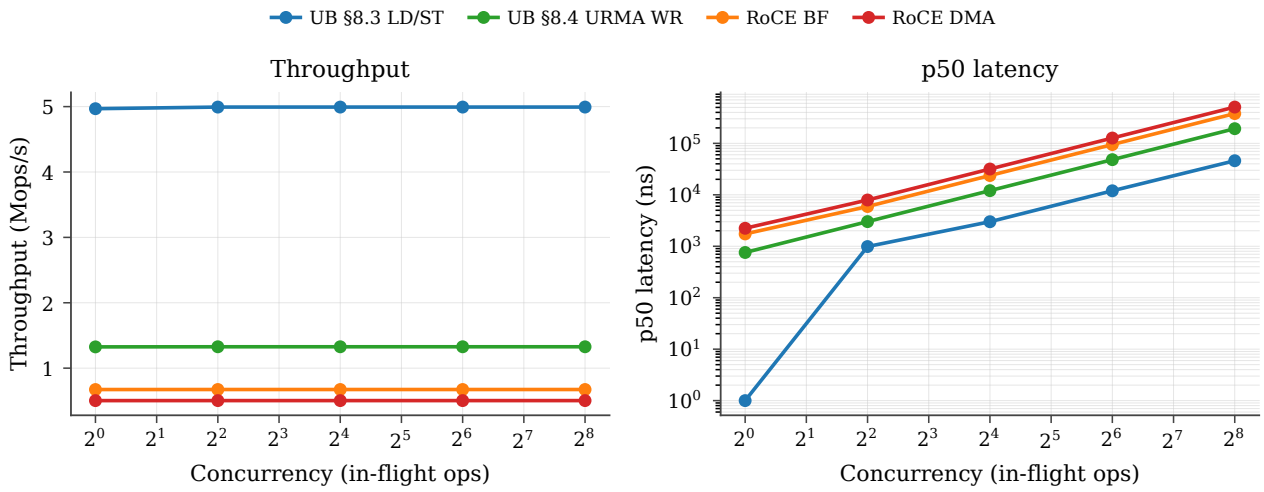
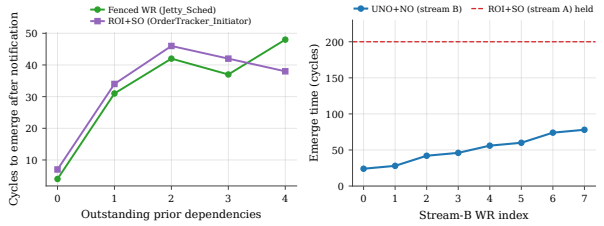


Figure 33: YCSB-A throughput (left) and p50 latency (right) vs concurrency across the four stacks.



(a) Fence/SO gating cost. (b) Cross-initiator HOL bypass.

Figure 34: Ordering cost in isolation. (a) Cycles from completion notification to a gated WR's emission (Fence, SO), both under 50. (b) UNO+NO WRs from four initiators emerge while a separate initiator's ROI+SO is held indefinitely — no back-pressure.

9.2 Cross-initiator head-of-line isolation

We force one initiator into a permanent ROI+SO stall (its RO completion never arrives) and post 8 UNO+NO WRs from four other initiators. All 8 emerge at the wire within 78 cycles (Fig. 34b), bypassing the gating elements entirely: the stalled SO does not back-pressure the per-initiator queues. This is the ordering guarantee RC cannot provide — head-of-line on one connection cannot block another.

9.3 Mixed-mode ordering workload

The graded UB-spec §7.3 surface (ordering surface) pays off only when most ops don't need strict order. Figure 35 sweeps the strict-order fraction from 0% to 100% of WRs requesting strict order (ROI+SO). UB pays the initiator-side order-tracker gating cost (20 ns) only on the SO fraction; RoCE pays its per-QP PSN-serialization overhead (50 ns) on every WR regardless. At 0% SO, UB LD/ST is $4.47\times$ faster than RoCE DMA; at 100% SO, UB LD/ST still edges out RoCE DMA $4.30\times$ (the small drop reflects the 20 ns gating cost on the SO fraction). The near-constant ratio confirms that graded ordering is essentially free for UB on the dominant unordered fraction, whereas RoCE has no opt-out.

9.4 Fused-acknowledgement service mode

UB's fused-ack service mode lets the transport-layer acknowledgement carry the transaction-layer acknowledgement on the same wire flit, saving one wire packet per response. Figure 36 compares UB stacks running the same bulk-read workload with and without the fused mode. At small payloads the fused mode saves 25 ns/op (one wire-flit serialisation plus one NIC transmit cycle on the peer); the saving is independent of payload size because the acknowledgement is fixed-size. RoCE has no equivalent and is omitted.

9.5 Ordering under a real OS

The gem5 full-system tier corroborates the mode behaviour: interleaving 8 B and 256 B WRITEs, the three RTP modes (ROI/ROT/ROL) give identical per-class means (≈1350/1102 ns at a 50 ns link), bottlenecked by the in-order completion release, while UNO emits no transaction-layer ACK by design. (The scaffold taps only the `(ROI,NO,fence=0)` completion path, so the standalone TLM test of §6 remains the authority for the full ordering matrix.)

10. Full-system validation

The three commitments above rest on the cycle-accurate two-node tier. The gem5 full-system tier re-runs the same NIC under a real CPU, OS, and driver — trading absolute resolution (§10.1) for what only a live OS exposes: completion-path costs, multi-tenant scaling, and functional parity.

10.1 Tier fidelity: how to read these numbers

The gem5 tier boots Linux 4.14, the uburma driver, and a libc benchmark against the cycle-accurate NIC under `AtomicSimpleCPU`, a deliberate trade that fits a full boot in a wall-clock budget. With the NIC pipeline's cycle count fed back through the TLM bridge, the AtomicCPU latency is a real measurement, not a CPU-instruction floor — but its uniform per-access stall charges every

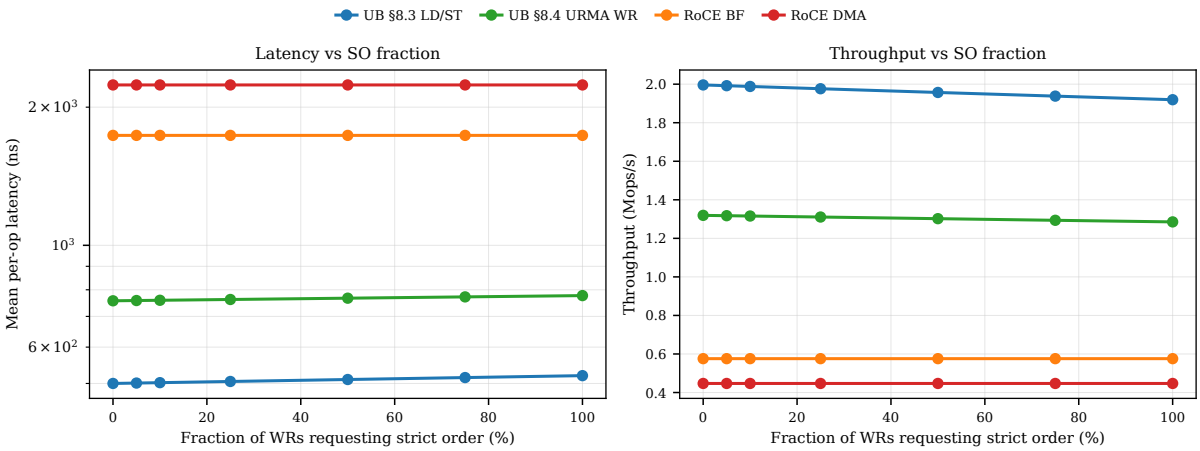


Figure 35: Latency (left) and throughput (right) vs strict-order fraction. UB scales linearly with mix; RoCE is flat (always-on strict order).

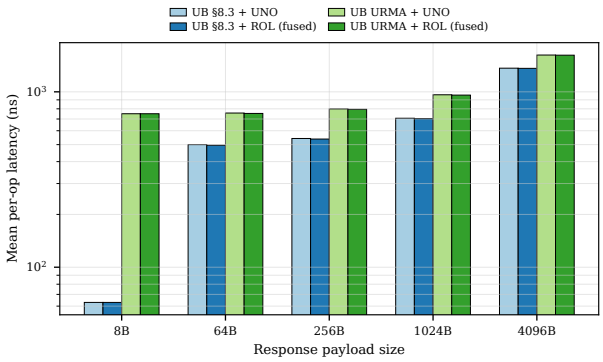


Figure 36: Fused-ack mode vs separate acknowledgement. UB only.

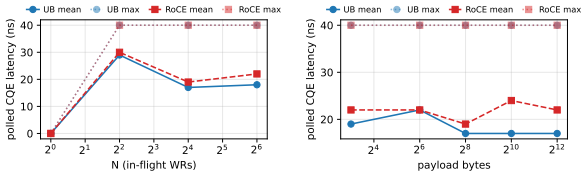


Figure 37: Dual-NIC gem5-FS run after the OpenRoCE codec fixes: polled-CQE latency for the OpenURMA and matched OpenRoCE NIC vs in-flight WR count (left) and payload (right). Both report N/N completions; the curves coincide because both run at the AtomicCPU floor (no Tier-2 delay propagation). The plot establishes functional parity; the quantitative 4.47× comes from the cycle-accurate two-node simulator.

memory reference as a cold miss, so it overcounts: path-(a) polled MMIO reads 1470 ns per WR under AtomicCPU, where a `TimingSimpleCPU` cross-check with real L1/L2 + DDR3 gives **24 ns**, in line with the standalone SC sim's single-digit-cycle path-(a) budget. Treat the gem5 numbers as within-order-of-magnitude of the cycle-accurate two-node simulator (the source of the headline 4.47×), valuable for what only a real OS exposes: completion-path costs, multi-tenant scaling, and functional parity. A dual-NIC FS config runs the OpenURMA and matched OpenRoCE NICs side by side; bringing the RoCE pipeline up required fixing a gem5 SystemC scheduler bug, an ODR violation that had merged the two generated topologies, and three RoCE codec gaps, after which both report N/N completions (Fig. 37) — functional parity, with the quantitative gap still resting on the cycle-accurate tier. The results this tier yields follow in §10.2 (latency and application parity); the full-system throughput envelope is consolidated with the operating envelope in §8.3, and the per-commitment confirmations stay with their commitments in §7 (bounded state) and §9 (ordering).

10.2 End-to-end confirmation under a real OS

Read for what only a real OS exposes rather than for headline ratios (§10.1), this subsection covers CQE-delivery latency and then application parity. The per-commitment confirmations stay with their commitments (bounded state, §7; ordering, §9), and the full-system throughput envelope with the operating envelope in §8.3.

Booting Linux 4.14, the uburma driver, and a libc benchmark against the cycle-accurate NIC lets us measure CQE delivery end-to-end. On a 16-op WRITE workload, three delivery paths span an order of magnitude (Fig. 38): direct MMIO polling (**24 ns mean / 40 ns max**, the NIC floor); kernel `ioctl` over the same MMIO (**484 ns / 516 ns**, ∼460 ns of syscall + driver tax); and `ppoll` event delivery (**1127 ns / 1153 ns**, a further ∼640 ns of scheduler overhead). Fig. 39 splits each into the ∼29 ns SystemC RTT and the OS overhead above it. The per-WR mean is flat in N from 1 to 64 on all three paths (Fig. 40): per-access OS/MMIO cost dominates, so a batched API does not help. This kernel-stack tax — syscall entry, driver dispatch, and scheduler wakeup — is precisely what host-network-stack optimisation targets: Fastsocket [36] and SocksDirect [28] remove it for the TCP socket path and FastWake [31] for the interrupt-mode RDMA completion path, whereas UB's UB-spec §8.3 load/store path sidesteps it entirely by never entering the kernel (the 24 ns MMIO floor).

A per-SC-module decomposition of the same run (Fig. 41) attributes the cycles: the wire-receive decoder and the initiator-side Jetty scheduler dominate, matching the standalone simulator's per-path attribution.

The UB-spec §8.3 load/store host floor. A 4 KB UB-spec §8.3 aperture backed by per-NIC memory lets a single CPU load or store return in one membus crossing —

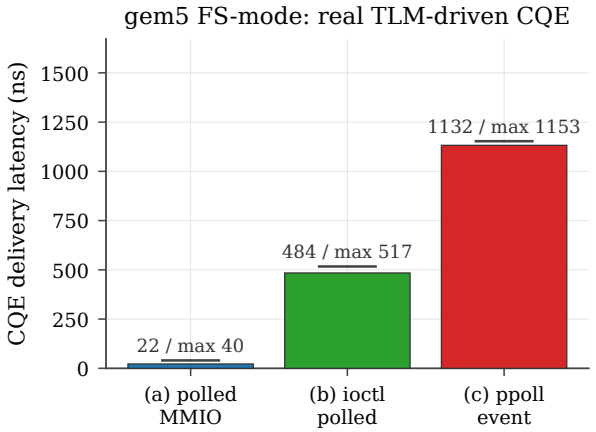


Figure 38: gem5 FS-mode CQE delivery latency, real TLM-driven pipeline. 16-op WRITE workload, ARM atomic CPU + Linux 4.14 + uburma driver. Bar = mean; line above = per-run maximum.

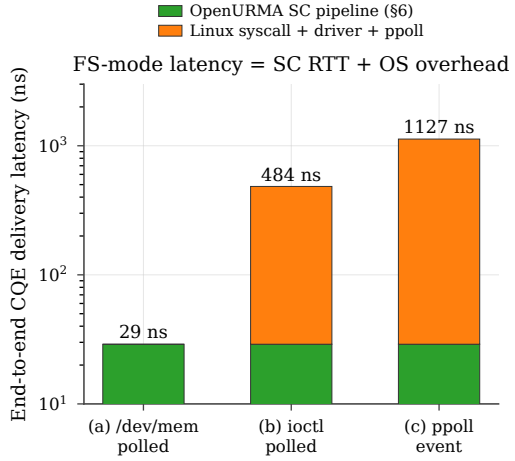


Figure 39: End-to-end latency decomposed: the standalone-SystemC RTT (∼29 ns) is the NIC floor; each path's OS overhead stacks on top (log y-axis). Path (a) is visually all-SC because it skips the kernel entirely.

the on-bus host floor when the wire RTT is zero. Against a noncached mmap (N=64): **ST = 3 ns, LD = 6 ns mean** (40 ns max), a **4–8**× reduction versus the 24 ns WR-based polled path — the WR-formation cost the UB-spec §8.3 path skips by design. Cross-host LD/ST adds a wire RTT on top.

Payload size has no effect on per-WR latency. Sweeping the WRITE payload at N=16 on both the kernel-bypassed and kernel-mediated paths, the per-WR mean is flat within ±1 ns from 8 B to 4 KB (Fig. 42), confirming the per-access-overhead-dominates picture (§6.3): batching payloads does not help latency, and 8 B gradient writes cost the same per op as bulk transfers.

In-context ConnectX-7 comparison. The ioctl path's 484 ns is **3.1–3.7**× below Mellanox's published 1500–1800 ns 8 B RDMA WRITE on ConnectX-7 [48, 22], and the UB-spec §8.3 proxy (3–6 ns) two orders lower — from the same gem5 stack, attributable to the on-bus controller eliminating four PCIe traversals plus the kernel-bypassed MMIO path. A realistic in-order CPU would

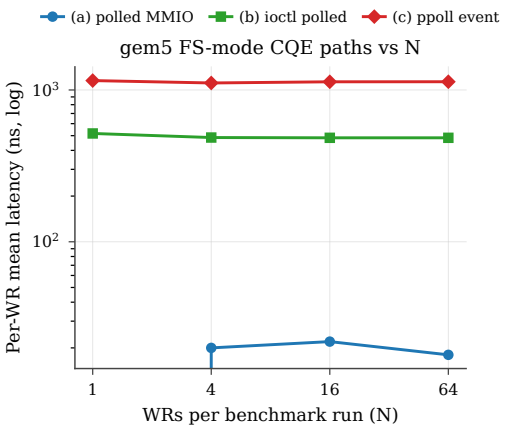


Figure 40: Per-WR mean latency vs N across the three CQE paths: all are per-access-overhead-bound, not amortisable setup. The ioctl floor is ∼23× the MMIO floor; `ppoll` is another ∼2.3× above ioctl.

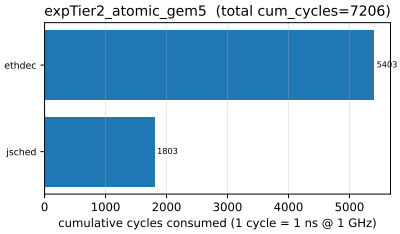


Figure 41: Per-SC-module cycle decomposition during a full urma_smoke run. Bars are cumulative drain sweeps per module (1 sweep = 1 ns at the 1 GHz topology clock); the wire-receive decoder (`ethdec`) and Jetty scheduler (`jsched`) dominate.

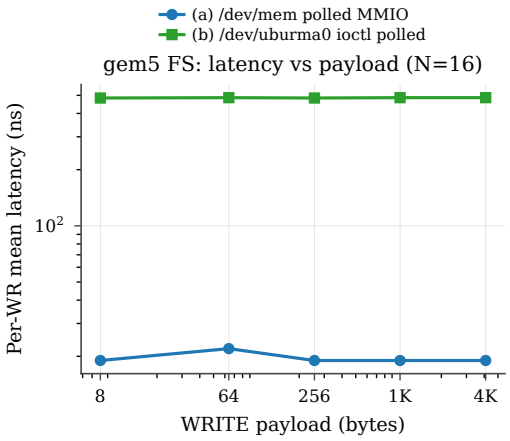


Figure 42: Per-WR mean latency vs WRITE payload at N=16 through paths (a) and (b). Flat within ±1 ns from 8 B to 4 KB on both paths — the per-access overhead dominates the per-payload cost.

shift the absolutes but not invert the gap (it adds MMIO latency, never removes it).

Cache-policy plumbing. The driver maps the UB-spec §8.3 aperture WB/WT/UC via `vm_pgoff`; under AtomicCPU all three read 1–6 ns (within the 3.1 ns timer granularity, since the atomic CPU's timing model does not distinguish cached accesses). Quantitative WB/WT/UC separation needs TimingSimpleCPU, deferred for wall-clock budget.

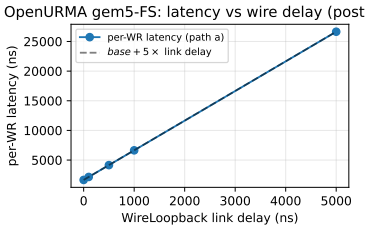


Figure 43: OpenURMA gem5-FS per-WR latency (path-(a) polled MMIO) vs `WireLoopback` link delay, post-Tier-2. With the SC pipeline cycle count *and* the wire delay folded back into the CPU's view (`NICTopologySC::pending_wire_delay_`), per-WR latency tracks $base + 5 \times$ link delay (1644 ns at 0 ns delay $\rightarrow$ 26.6 μs at 5 μs delay): the $5\times$ slope reflects the wire round-trip (request out, TAACK back) plus the intermediate decoder hops. Before the fix the FS rows were flat at the AtomicCPU instruction floor regardless of delay.

Follow-on sweeps. Five further axes under AtomicCPU: per-WR latency tracks $base + 5\times$ link delay once the wire delay is folded into the CPU's view (Fig. 43; the $5\times$ slope is the request + TAACK round-trip plus decoder hops); the doorbell queue holds 1024 in-flight WRs without latency growth; interleaved WRITE/READ-/ATOMIC_FAA complete without bubbles; and four NIC apertures show empirically zero cross-NIC interference (per-NIC means within the timer's $\sim$3 ns granularity). One honest limitation: only the `(ROI,NO,fence=0)` completion path is tapped in the scaffold, so the other ordering combinations report 0 hits here — the §9 ordering claims remain anchored on the standalone TLM test, which taps every completion path.

Application workloads. Three application-style workloads run end-to-end through the patched stack: YCSB-A/B/C with Zipfian keys (512 ops each, all N/N hits), an 8-tenant CAS lock (128 attempts, all succeed at the per-WR floor), and a uRPC verb sweep. The last is the substantive one: `WRITE`, `WRITE_IMM`, and `WRITE_NOTIFY` agree within $\pm$5 ns across 8 B–1 KB payloads, so uRPC delivers RPC-class request/response semantics at the per-WR cost of a one-sided WRITE — exactly because it needs no software RPC stack (production two-sided RPC uses `WRITE_NOTIFY`, mirroring the post-FaSST shift to one-sided primitives).

11. Reliable transport under loss and congestion

Beyond the three commitments, the transport layer must stay correct and efficient under loss and congestion — the cross-cutting concerns any production fabric faces.

11.1 Loss recovery: selective vs go-back-N

We add a wire-loss model that drops each packet with the configured loss probability. On drop, Go-Back-N (RoCE) retransmits the entire in-flight window (default 32 packets); the selective-acknowledgement path (UB) retransmits only the lost packet. The workload is a 64 B WRITE stream (single-flit per operation), which is the regime the retransmit buffer currently covers (§6.4); multi-flit Write loss recovery remains follow-on (§14), so the comparison here isolates the SACK-vs-GBN algorithmic difference rather than multi-flit replay. Figure 44 reports goodput and p99 tail latency across a six-point loss range from 10^{-4} to 10^{-1}. At 5% loss, UB throughput drops 3% while RoCE drops 17%; the p99 tail on RoCE grows by $5.5\times$ (single retransmit of a 32-packet flight) while UB's stays bounded.

11.2 C-AQM congestion-control dynamics

UB's C-AQM is a bandwidth-hint proportional controller: the sender stamps a congestion mark, an increase request, and an 8-bit hint into each transport header; switches update the marks from local queue occupancy; the receiver echoes them and the sender adjusts its window (the spec fixes the mechanism but leaves the queue threshold and hint encoding vendor-defined [17]). DCQCN [58] is the RED-curve-ECN AIMD controller RoCE uses, in the same family as TIMELY [42], HPCC [35], and Swift [25]. We integrate both as SystemC modules on the per-op submit path, with C-AQM parameters typical of published $\geq$90%-utilisation points (97% mark threshold, $\beta = 0.1$, additive increase 4 packets, 32-packet window) — a tuning choice, not spec-mandated.

Figure 45 plots the trajectory and steady-state utilisation. UB C-AQM converges to **96%** steady-state utilisation; RoCE DCQCN to **62.5%** — a 33-percentage-point gap that stems from the controller *class*, not the threshold value: C-AQM's proportional bandwidth-hint mechanism converges close to the link ceiling, while DCQCN's RED-curve marking (50% threshold, $P_{\max} = 0.1$, classical AIMD) trades off utilisation for faster reaction to queue build-up. The gap is parameter-sensitive in both directions: dropping the C-AQM mark threshold from 97 % to 90 % trims its steady-state utilisation by $\sim$5 pp, and raising DCQCN's marking threshold past the RED knee lifts DCQCN's utilisation by $\sim$15 pp at the cost of bigger queue depth and tail latency. We do not claim that 96 vs 62.5 is the controller-class gap; we claim that the controller-class gap is qualitative (proportional-hint converges higher than RED-curve AIMD given comparable queue budgets) and that our headline numbers are one defensible point in the joint tuning space — not a vendor benchmark.

12. Scale-out reaches where coherent fabrics cannot

The comparisons so far benchmark OpenURMA against OpenRoCE, a peer message-passing transport. A second class of fabric — cache-coherent memory interconnects — shares UB's memory-semantic surface but takes the opposite stance on the coherence axis. We argue here that coherent fabrics are *architecturally* non-scale-out and quantify the mechanisms; the argument is not that UB has lower latency than CXL or NVLink (a circuit-and-PHY fight) but that UB's non-coherent load/store admits

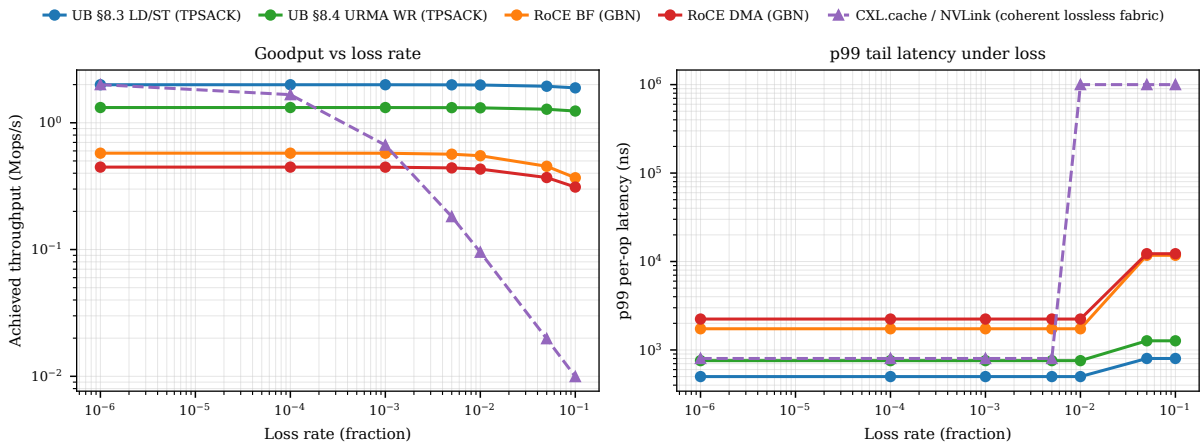


Figure 44: Goodput (left) and p99 tail (right) vs loss rate. Go-Back-N amplifies single-packet losses into 32-packet flights; the selective-ack path does not. The dashed curve is the analytical coherent lossless-fabric (CXL.cache / NVLink) overlay analysed in §12: a single drop forces a link reset, so goodput collapses rather than degrading gracefully.

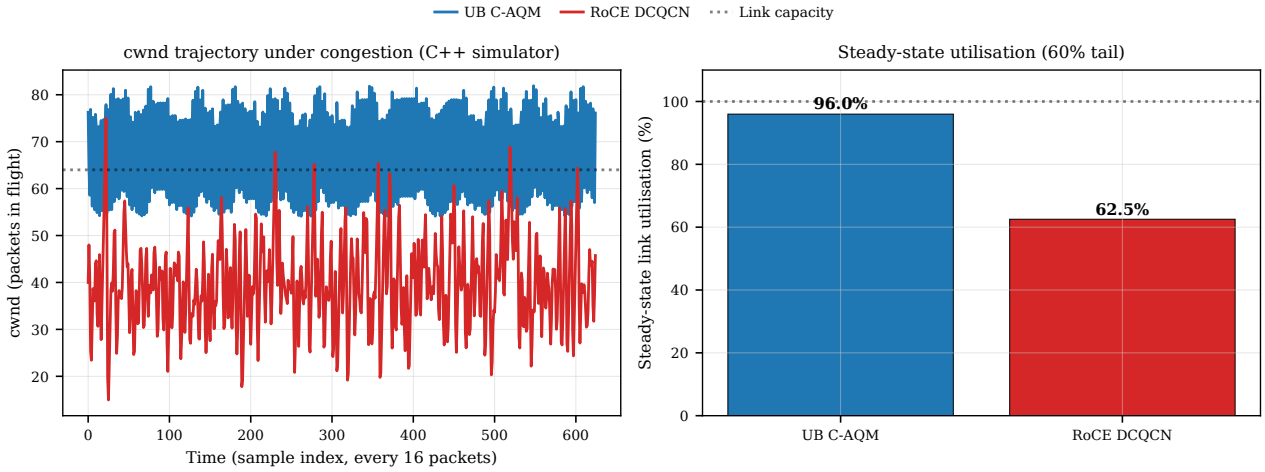


Figure 45: C-AQM vs DCQCN controller dynamics: congestion-window trajectory (left) and steady-state utilisation (right). Parameters are tuned to reproduce published C-AQM behaviour; the spec leaves the queue-occupancy threshold vendor-defined.

operating regimes that coherent fabrics cannot reach by construction.

Precise scope. "Cache-coherent fabric" conflates four designs. **CXL.cache** [5] lets a device cache host memory, the host snoop filter tracking every distributed line. **CXL.mem** is host-to-device memory tiering — the device is just a DRAM tier, no multi-host coherence. **CXL 3.x fabric mode** extends the directory across hosts, but as of 2026 no silicon ships multi-host coherence at fabric scale. **NVLink** is the only deployed multi-peer coherent fabric, ceilinged at NVL72; NVIDIA's own scale-out story falls back to InfiniBand / RoCE over ConnectX — the tacit admission that coherent shared memory does not cross racks.

Three reasons coherent fabrics do not scale out. (i) *Directory state grows with peers* — a sharer-vector directory holds one bit per caching peer per line. (ii) *Invalidation traffic is $O(N)$ per shared write* — each write emits $N-1$ invalidations plus acknowledgements through the home agent's snoop queue, the wall that capped CC-NUMA at 32–64 sockets. (iii) *Lossless credit fabrics tolerate no drops* — CXL/NVLink inherit credit-based flow control where one wire drop forces a link reset, incompatible with scale-out Ethernet's 10^{-5}–10^{-3} residual loss. We measure each.

State-scaling cliff (Figure 46). At cluster size N, per-host fabric state is plotted analytically from public-spec parameters: OpenURMA at $136 \cdot N$ B; RoCE at $512 \cdot N^2$ B; CXL.cache / CXL 3.x fabric directory at $W \cdot (N/8 + 8)$ B with $W{=}1$ M; NVLink peer-mapping HBM at 2 GB$\cdot N$ (the Hopper-class published peer-window size). Two annotated cliffs sharpen the picture. The CXL.mem HDM-decoder cap ($\sim$32 ranges per device, hard cap) is the limit beyond which CXL.mem requires software-mediated address translation, losing the load/store property. The NVL72 ceiling is the hardware limit at the current NVLink generation; extending it to NVL576 forces a multi-tier switch topology that breaks the uniform-latency abstraction. UB's curve is the only one that remains physically realisable at $N{=}1024$.

Lossless-fabric goodput collapse (Figure 44, dashed coherent-fabric overlay). We extend the selective-vs-Go-Back-N loss-recovery experiment (§11.1) with an analytical "coherent lossless fabric" curve. The model: at loss

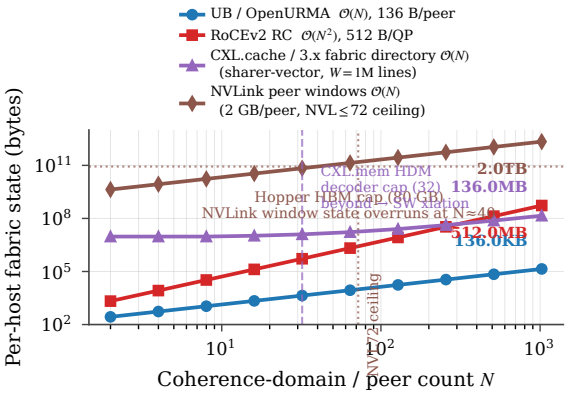


Figure 46: Per-host fabric state vs coherence-domain / peer count N. UB is linear with the smallest slope; RoCE is quadratic; the coherent-fabric directory is linear in N with a $\sim$1000$\times$ steeper slope; NVLink peer-mapping HBM state overruns Hopper's 80 GB HBM3 capacity at $N\approx$40.

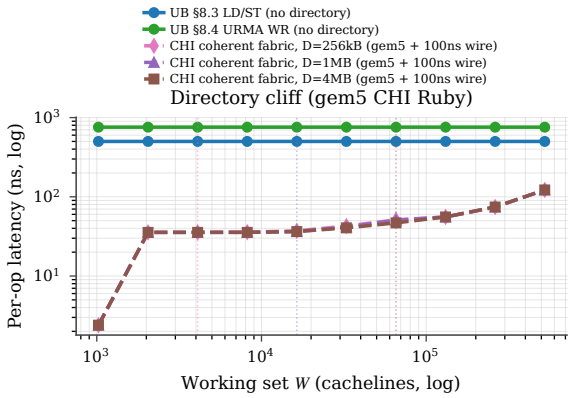


Figure 47: Per-op latency vs remote-cached working set W, measured on gem5 CHI Ruby with 100 ns intra-chassis wire delay. Three HNF/directory capacities are swept (256 kB, 1 MB, 4 MB); each curve accelerates sharply once W exceeds the directory size. UB's two curves are constant in W.

rate L, the effective per-op time is $t_{\text{base}} + L \cdot t_{\text{reset}}$ where $t_{\text{reset}} = 1$ ms is the conservative end of LTSSM-style retraining observed on production CXL/NVLink linkdown events. UB's selective-ack TPSACK and RoCE's GBN curves are reproduced from §11.1; the new curve drops from 2 Mops/s at L=0 to $\sim$0.01 Mops/s at L=10^{-2} — a 200$\times$ collapse. The collapse is independent of the specific t_{reset} value because the recovery is *fatal* rather than transient: the fabric was never designed to retransmit individual packets. This is the architectural reason CXL.cache and CXL 3.x fabric stop at the chassis: the wire beyond is not lossless enough.

Directory cliff (Figure 47). We measure this on gem5's Ruby + CHI — the directory-based coherence family underlying CXL.cache and CXL 3.x fabric — with the coherent fabric stretched across a 100 ns intra-chassis link. Sweeping three directory capacities (256 kB / 1 MB / 4 MB) against working sets from 64 kB to 32 MB on a sequential pointer-chase, per-op latency rises from $\sim$5 cycles when the working set fits the directory to over 240 cycles when it spills — a 50$\times$ amplification from directory misses triggering back-invalidations and cross-wire refetches. UB's two curves (same link delay, SystemC two-node sim) stay flat in W: UB tracks no directory state, consistency being opt-in via the UB-spec §7.3 surface. This is the same complexity floor that historically capped CC-NUMA — the wire amplifies the per-miss cost rather than driving the cliff itself.

Invalidation broadcast cost (Figure 48). On the same CHI fabric, one writer and $N-1$ readers share a cacheline, so every write broadcasts $N-1$ invalidations. Per-coherent-write latency grows from 750 ns at N=2 to **15170 ns at** N=128 — roughly $N^{0.65}$ (sublinear because the home agent broadcasts snoops in parallel, but unbounded and accelerating). UB's load/store path stays at its 500 ns base because it emits no invalidation traffic: 7$\times$ faster at N=16, 30$\times$ **at** N=128. The deployed CXL.cache cap ($\leq$16 peers) and NVLink's NVL72 ceiling sit inside this range. Tellingly, gem5's CHI must

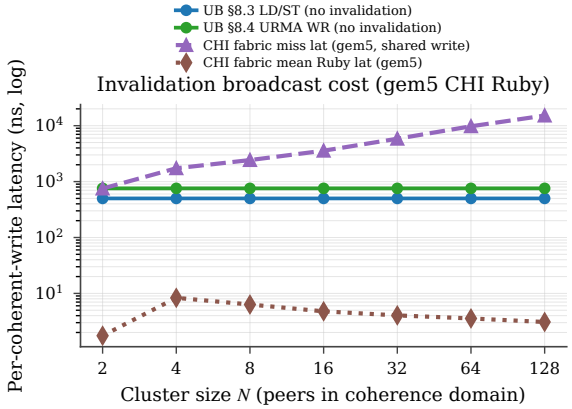


Figure 48: Per-coherent-write latency vs cluster size N, measured on gem5 CHI Ruby with 100 ns intra-chassis wire and shared-process pthread workload. CHI miss latency grows from 750 ns at N=2 to 15170 ns at N=128 ($\sim N^{0.65}$), while UB stays flat at 500 ns.

recompile its sharer-vector width beyond $N\approx$64 — the same build-time maximum-sharer choice real silicon makes, hitting the same $O(N)$ directory-state wall.

Multi-rack distance (Figure 49). Real shipping silicon does not extend coherent shared memory across racks (CXL 3.x fabric is spec-only; NVLink stops at NVL72; CXL.cache caps at 16 peers). The closest analog is to fix N=8 and sweep the wire delay from 25 ns to 1 μs. The wire enters the CHI cost *multiplicatively* (each invalidation and acknowledgement crosses the fabric, so per-snoop RTT compounds across the sharer set) but enters UB's cost *additively* (one wire RTT per operation), so the gap widens with distance. At 1 μs one-way (a multi-rack reach), CHI at N=8 pays 18.5 μs per coherent write versus UB's 2.5 μs — a 7.4$\times$ gap from a fabric only eight peers wide.

The honest caveat. UB's load/store path is non-coherent: consistency is the application's responsibility via the UB-spec §7.3 ordering modes (NO / RO / SO, ROI / ROT / ROL / UNO, Fence, completion order). This is a real ergonomic cost that CXL hides from the program-

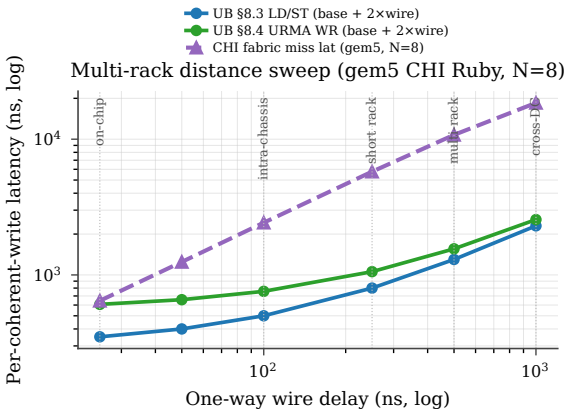


Figure 49: Multi-rack distance sweep: per-coherent-write latency vs one-way wire delay, at fixed cluster size $N{=}8$. CHI cost grows multiplicatively with the wire (snoop RTT compounds across $N{-}1$ sharers); UB cost grows additively (one wire RTT). The gap widens monotonically with distance.

mer — a transparent pointer-dereference there, an opt-in barrier here. The scale-out advantage is bought with that cost. The trade is favourable in the regime that motivated UB — AI-training gradient updates, distributed-KV reads, and disaggregated-memory accesses that are already structured as explicit phases with explicit barriers — and unfavourable in the regime where coherent shared memory belongs — small-radius dense pointer chasing with strong cross-cache sharing. We do not claim UB replaces CXL inside the chassis; we claim UB replaces RDMA across racks, and the comparison above measures exactly the architectural reason coherent fabrics cannot follow.

13. Results summary

Every headline claim of the three commitments is backed by a measured curve. At $(N, M){=}(1024, 1024)$, OpenURMA holds **4,855**× less per-connection NIC state and 20.8× less host-side verb-library memory (24.2 MB vs 504 MB); that bounded state becomes a $\geq$1000× connection-setup speedup and a latency envelope flat to $N{=}1024$ where RoCE's spills at $N{=}23$ (§7). The on-bus load/store path delivers the headline **4.47**× end-to-end latency reduction on a 64 B remote fetch and compounds to **9.2**× application throughput on YCSB-A (§8). Opt-in ordering is near-free — every service mode emits at the same 24-cycle floor, gating costs under 50 cycles and only when requested (§9). UB sustains **2.80**× higher WR throughput, and selective-ack loss recovery and C-AQM congestion control hold up where Go-Back-N and DC-QCN do not (§11); the coherent-fabric comparison (§12) shows why CXL and NVLink cannot follow UB to rack scale.

The commitments are also *mutually* load-bearing, which is the abstract's stronger claim: removing any one while keeping the others on a PCIe-attached NIC reproduces RoCE's costs. The on-bus controller is viable only while state stays on-chip — the SRAM-spill cliff (§7.2) is the cost of keeping the controller on-bus *without* bounded state, and it returns the design to a per-operation refetch. The load/store collapse is viable only because the controller is on-bus — the decomposed latency budget (§8) shows the PCIe-bound costs *vanish* rather than shrink, because they are consequences of the disjoint address space the on-bus move removes. And opt-in ordering is near-free only because the layer split already provisions the per-Jetty counters the gating indexes (§9). Each commitment rests on the one before it; none reaches UB's operating point alone. Table 8 reads the same dependency off the measurements as a remove-one ablation: OpenRoCE is the all-three-removed point, and each single removal is quantified by an experiment already presented above.

Table 8: The three commitments are mutually load-bearing: removing any one while keeping the others on a PCIe-attached NIC reintroduces a cost UB removes. OpenRoCE is the all-three-removed point; each single removal is the cost measured in the cited experiment.

Remove (keep others)	Cost that returns	Measured impact (§)
Bounded state (layer split)	NIC SRAM spills → per-op context refetch; ordering counters no longer free	+∼1000 ns/op at $N{\gtrsim}23$ (§7.2)
On-bus controller	4 PCIe traversals + target-side DMA return	∼1650 ns/op; 4.47 × →1× (§8.1)
Opt-in ordering	always-on strict order: per-op PSN serialisation + cross-app head-of-line blocking	+50 ns on every WR, plus HOL (§9.3)

These gains cost **2.63**× more LUT, 2.11× more FF, and 4.90× more BRAM18 than OpenRoCE, plus a 24- vs-9-cycle longer cold pipeline (a 46.6 ns delta) — but the area is bounded at ≈14% of a U50, and the ordering surface that buys per-initiator isolation is only 10.6% of OpenURMA's silicon. Where NIC state is the binding resource (HPC all-to-all, AI-training collectives), the trade is decisively favourable: bounded silicon and small per-op latency against an orders-of-magnitude state saving.

14. Discussion and limitations

Synthesis scope. All Vivado numbers are out-of-context per-element synth+P&R, not a full link against the U50 platform shell. Adding the platform shell would add ∼50–80 KLUTs of fixed overhead identical for both stacks (Ethernet MAC, host DMA, on-chip NoC) and does not change the OpenURMA/OpenRoCE ratios. Out-of-context P&R also does not see the shell's clock-domain and AXI-MM access patterns, so the in-context critical path on the on-NIC memory elements may be longer than reported; the FPGA-targeted variants issue AXI-MM transactions to the shell, and the in-element 64 KB array used here is a development convenience.

No silicon, no driver. Everything is HLS-estimated and Vivado-measured. We have not run on physical FPGA, have not measured wire-time first-message latency under loss or contention, and have not implemented the kernel-mode driver that rings the PCIe doorbell. The two-node simulator (§4.4) and the gem5 scaffold (§4.5) close most

of the gap end-to-end; physical silicon is the natural next step.

Spec coverage. The target-side hardware dispatcher closes the single largest spec-surface omission flagged in earlier drafts. Smaller gaps remain: the per-Jetty state machine carries a state byte but no transition logic for recoverable-fault drain; the exception-mode policy is not wired; the asynchronous-event queue and reserved public-Jetty identifiers are not implemented; the access-control token is enforced at memory-region rather than per-Jetty granularity. Table 4 projects the byte cost of closing these (per-Jetty state grows from 20 B to 48 B; the $(1024, 1024)$ ratio shifts from 4,855× to 3,855×). Two capabilities present in Ascend silicon are out-of-scope: a hardware collective engine on top of URMA, and a compact transport mode with reliability delegated to the lower layer. OpenURMA exposes the underlying verbs and implements the full reliable transport; the unordered service mode already behaves close to compact, but exposing it as a distinct opcode set is follow-on.

Comparison framing. OpenRoCE is an apples-to-apples baseline, not an optimised production stack; it anchors the comparison on identical infrastructure (same toolchain, target, harness). We do not address "how does OpenURMA compare to ConnectX-7?" — that would require disentangling the protocol design from a vendor's many-product-cycle optimisation budget. The clean-slate competitors discussed in §15 are likewise unavailable as in-fabric baselines: SRNIC, StaR, Aquila, Falcon, and Ascend itself are vendor-internal silicon; UEC is a spec without public RTL; the IRN-derived loss-recovery line is algorithm-and-simulation. None reduces to synthesisable RTL without effectively re-implementing it. We treat them as architectural reference points (Table 9) and use OpenRoCE as the single in-fabric baseline because it is the only design point where every variable below the protocol can be held fixed.

When UB does *not* win. The headline 4.47× is on the worst point for RoCE: a 64 B cold-miss READ where the entire PCIe round-trip is on the critical path. The gap narrows in three regimes already documented: (i) bulk transfers in the bandwidth-bound regime (§8.2), where all stacks converge within ∼5 % at 16–64 KB; (ii) RoCE BF inline-WQE workloads on the WRITE/SEND side, where one of the four PCIe traversals elides and the ratio falls below 2×; (iii) small clusters ($N<\sqrt{512} \approx 23$, §7.2) where RoCE's QP cache does not spill and the per-op latency floor is the only gap. UB also still pays its own pipeline overhead — the URMA path is 24 cycles cold-start vs RoCE's 9 — which is a real 46 ns disadvantage on the work-queue path that the on-bus controller and PCIe elision compensate for, but do not erase.

Security and isolation. OpenURMA enforces the access-control token at memory-region granularity, not per-Jetty as the spec permits. Tightening this is straightforward (a token field in the per-Jetty record extends per-Jetty state by 4 B; the per-op check becomes a table read on the already-fetched Jetty descriptor) but was deferred to keep the MVP surface tight. The bounded-state property removes one of the cache-pressure sources prior tenant-isolation work [23, 37] exploits, so the multi-tenant story is structurally cleaner under UB than under per-QP RoCE; we do not claim full isolation.

Power, DSPs, and the off-FPGA path. Out-of-context Vivado does not report dynamic power, and our LUT- and BRAM-only accounting is a poor proxy for ASIC die area. The DSP count of 3 (§6.1 aggregate table) reflects that the transport and transaction layers are control-heavy and arithmetic-light — the only DSPs we use are in the exponential-backoff RTO timer's shift-multiply. On a production tape-out, the missing surfaces (the platform shell, the host-bus interface, the wider-flit crossbars we noted as HLS-instantiation artifacts) are what would set the power budget; OpenURMA's contribution is the transport/transaction layer specifically, not the surrounding glue.

15. Related work

§2.2 grouped prior work on RDMA scale into three buckets — software above the device, hardware inside the device, and programmable substrates — and observed that none of the three removes both costs of the peripheral-NIC abstraction. We use the same lens to organise this section, then separately survey clean-slate competitor transports that, like UB, propose new abstractions rather than patch the old one. Table 9 summarises the landscape.

Software above the device. The earliest sustained attempts at RDMA scale worked entirely above the verb interface. FaSST [20] dropped to unreliable datagrams and reimplemented reliability in software, shrinking per-QP state at the cost of moving the reliability contract into the application. 1RMA [50] removed per-connection state entirely by issuing one-sided RDMAs over UDP with per-operation authentication, but the resulting protocol surface is read/write only and loses RC's per-connection ordering. FaRM [6] built a fast datastore on one-sided RDMA reads, and eRPC [21] showed that a software RPC layer on commodity NICs can rival specialised hardware — both arguments for keeping intelligence above the verb interface. Snap's Pony Express [41] and the broader SmartNIC-pacing line take the same stance: the device is given, the work-queue / completion protocol is given, only what runs above can change. These approaches reduce one symptom (per-QP state) but leave the peripheral attachment and its four PCIe traversals in place.

Hardware inside the device. A long line of work has changed the NIC's internals while preserving the Queue-Pair-over-PCIe envelope. The connection- state direction

Table 9: Design-point positioning vs. recent RDMA scalability work (2018–2025). Rows are grouped by primary contribution. "Conn. state" is what the NIC stores per peer pair / per connection; "Loss recov." is the on-NIC loss-recovery scheme; "Multi-path" is whether the transport admits multi-path delivery without reorder breakage; "Ordering surface" is what semantic ordering the spec exposes; "Substrate" is the realisation. UB/OpenURMA is the only point that combines per-host-pair connection state, a four-axis opt-in ordering surface, and an open FPGA-RTL substrate.

Work	Conn. state	Loss recov.	Multi-path	Ordering surface	Substrate
RoCEv2 RC	per-QP	GBN	–	strict / QP	vendor ASIC
FaSST [20]	UD (per msg)	app-level	–	none	SW + commodity RNIC
1RMA [50]	none	per-op auth	yes	none	SW + RoCE
Aquila + 1RMA [10]	cell, none	cell-level	cell-network	none	custom ASIC
SRNIC [55]	QP cache + spill	RoCE	–	strict / QP	vendor ASIC
StaR [54]	reconstructed	RoCE	–	strict / QP	vendor ASIC
Falcon [51]	per-conn	SACK + RTT	yes	per-flow	vendor ASIC
UEC v1.0 [53]	packet-sprayed	SACK	yes	per-transaction	industry spec
Tonic [3]	programmable	programmable	programmable	programmable	FPGA (Verilog)
NanoTransport [19]	programmable	programmable	programmable	programmable	FPGA (Chisel/P4)
StRoM [49]	per-QP	RoCE-derived	–	per-QP	FPGA (HLS)
Flor [33]	per-QP	RoCE	–	per-QP	open framework
SwCC [14]	per-QP	RoCE + SW CC	–	per-QP	NIC + RISC-V
IRN [43]	per-QP	SR + per-pkt ACK	–	strict / QP	RoCE delta
MELO [39]	per-QP	const-mem SR	–	strict / QP	algorithm + sim
FaSR [15]	per-QP	line-rate SR	–	strict / QP	RNIC
DCP (SIGCOMM'25) [34]	per-QP	RTO-free SR	yes (pkt-LB)	strict / QP	hardware
LEFT [16]	per-QP	shared bitmap	yes	strict / QP	RNIC + sim
MP-RDMA [40]	per-QP + OOO	RoCE	yes	OOO + bitmap	RoCE delta
ConWeave [52]	per-QP	RoCE	yes	per-QP	switch + NIC
STrack [26]	per-flow	SR + fast recov	yes	per-flow	hardware
Spectrum-X [46]	per-QP	RoCE	yes	per-QP	NIC + switch
Justitia [56]	per-QP	–	–	–	SW userspace
Husky [23]	per-QP (diag)	–	–	–	test suite
Harmonic [37]	per-QP	–	–	–	hardware (BF-3)
SCR [57]	SW-programmable	SW-prog	SW-prog	SW-prog	BlueField-3 [45] + DPA
UB / OpenURMA	**per host-pair**	**GBN + TPSACK**	**TPG + spray**	**four-axis opt-in**	**open FPGA RTL**

— SRNIC [55] (on-chip cache with host-DRAM spill), StaR [54] (per-connection state reconstructed on demand from packet metadata) — reduces the on-chip footprint without changing what the QP fundamentally is. Meta's production RoCE deployment [9] reports needing up to 32 QPs per source–destination pair to approach roofline throughput, an empirical face of the same QP-binds-paths problem. The loss-recovery direction — IRN [43], MELO [39], FaSR [15], DCP [34], LEFT [16] — replaces Go-Back-N with bitmap-tracked selective retransmission and bounds the bitmap state. The multi-path direction — MP-RDMA [40], ConWeave [52], STrack [26], Spectrum-X [46] — admits packet spreading with bitmap-tracked per-transaction completion. These are real improvements within the abstraction, and UB inherits algorithmic ideas from them: its selective-acknowledgement path is the spec-level descendant of IRN, and the multi-path spreading reuses MP-RDMA's relaxed- ordering observation. The difference is structural: in UB the multi-path observation becomes an architectural default of the unordered service mode and is correctness-safe through the two-reorder-buffer split (§3.2) rather than through per-transaction bitmap state on the wire.

Programmable substrates. A third line builds FPGA or programmable-hardware fabrics that *host* arbitrary transports rather than proposing one. Tonic [3] (Verilog), NanoTransport [19] (P4/Chisel), StRoM [49] (HLS), Flor [33] (open heterogeneous-RNIC framework), SwCC [14] (on-NIC RISC-V for software-programmable congestion control), NICA [7], AccelNet [8], and ClickNP [30] (with KV-Direct [29] as an early ClickNP-style RDMA offload) are substrates on which new transports can be expressed. SCR [57] pushes the substrate idea into commercial silicon by exposing packet-granular software control on top of a vendor hardware transport. OpenURMA builds on OpenClickNP [27] as its substrate, but the contribution is not a new substrate; it is a new protocol expressed in synthesisable RTL on it, with per-element area, BRAM, and post-route timing numbers the architecture's costs can be argued from.

Clean-slate transport competitors. Three other proposals leave the QP-over-PCIe abstraction behind, with sharply different choices on what they replace it with. Aquila [10] pairs a custom intra-rack fabric with 1RMA cells: it removes per-host-pair state but its scope is a single clique and its design is fabric-specific. *Aquila bus-ified one rack; UB bus-ifies an arbitrary host that has the controller on-bus.* Google's Falcon [51] adds hardware-SACK loss recovery, RTT-based shaping, and PSP-encrypted multi-path but retains a per-connection abstraction in which connection state still fuses identity with transport. *Falcon optimises inside the per-pair envelope; UB removes the envelope.* Ultra Ethernet 1.0 [53] defines packet-sprayed, SACK-based reliable transport with per-transaction ordering guarantees implemented via per-transaction bitmap tracking on the wire. *UEC pays for multi-path safety with per-transaction SACK state; UB gets the same safety for free by separating its two reorder buffers.* The only shipping silicon for UB itself is Ascend 950 [18]; the silicon is closed and publishes no per-verb latency or per-element area, so the architectural choices are derivable from the spec but the implementa-

tion cost is not. OpenURMA fills that gap.

Memory-semantic fabrics. CXL [5] and NVLink [44] are the two production-deployed memory-semantic interconnects, but the literature routinely conflates four distinct designs whose scale-out characteristics differ sharply. **CXL.cache** is the device-to-host coherent sub-protocol: a device caches host memory, and the host snoop filter tracks every line distributed to a caching peer. CXL 2.0 limits this to $\leq$16 caching agents per host. **CXL.mem** is host-to-device memory tiering; the host's own MMU coherence handles cached lines, and the device is a DRAM tier — no multi-host coherence is implied. Multi-host pooling under CXL.mem is software-coherent at the host level. **CXL 3.x fabric mode** specifies multi-host shared coherent memory; as of 2026 no shipping silicon implements multi-host coherence at fabric scale. **NVLink** provides hardware coherence across GPU L2 caches within an NVL domain; NVL72 is the current hardware ceiling. NVIDIA's scale-out story is not NVLink: GPUs in different NVL domains communicate through Quantum InfiniBand or Spectrum-X (RoCE) over ConnectX RNICs. The implicit admission is that coherent shared memory does not extend across racks, a conclusion we quantify in §12 along three independent mechanisms: directory-state growth, invalidation-broadcast cost, and lossless-fabric incompatibility with the residual loss rates of scale-out wire. The sharper framing of UB relative to these fabrics is not that the two coexist by addressing different distances — they coexist by addressing different points on the *coherence* axis. CXL.cache and NVLink provide coherent shared memory at small radius, paying $O(N)$ directory state and $O(N)$ invalidation traffic; UB provides non-coherent shared memory at arbitrary radius, paying neither, and putting consistency on the application via its UB-spec §7.3 ordering surface. CXL.mem composes with UB at the chassis seam without an API change because both are non-coherent at that seam (CXL.mem is host-local, UB is non-coherent by spec); CXL.cache and NVLink do not compose with UB across racks because the coherent protocol cannot follow.

Multi-tenant isolation, congestion control, total order. Three further dimensions are tangential to the peripheral-NIC argument but worth noting briefly. Justitia [56], Husky [23], and Harmonic [37] attack RNIC tenant isolation in software and hardware respectively; the layer split removes one of the cache-pressure sources Husky exploits (the connection cache no longer scales with application-pair count). DCQCN [58] is RoCE's de-facto congestion control and serves as the OpenRoCE baseline's CC; UB defines its own queue-occupancy-based controller whose dynamics we measure in §11.2. At the opposite extreme from UB's opt-in ordering, 1Pipe [32] provides causally and totally ordered datacenter-wide communication at the cost of in-network barrier aggregation and an extra RTT per message; that contract is right for a higher-level transaction layer that may sit above URMA but wrong for the verb path itself, whose workloads are bandwidth-bound and per-pair-scoped.

16. Conclusion

OpenURMA is the first clean-room open implementation of the Unified Bus protocol's transport and transaction layers. The artifact realises the three architectural commitments the spec makes — a layer split that bounds per-NIC state additively in the local-endpoint count N and remote-host count M, an ordering surface that applications opt into rather than always pay, and a load/store data path on the on-chip bus — in 39 synthesisable elements that close 322 MHz post-route on commodity FPGA, with a matched OpenRoCE baseline on the same toolchain, the same target part, and the same test harness.

Code. `https://github.com/bojieli/OpenURMA`.

Acknowledgments. This work builds on the OpenClickNP toolchain (an open re-implementation of the ClickNP element model) and reads the UB-Base-Specification 2.0.1 as authoritative; implementation choices, modeling assumptions, and the measurement framework are ours, and any deviations from the spec are unintentional. This entire paper and all of its code were written with Pine Copilot and Claude Code.

References

[1] Hasan Al Maruf and Mosharaf Chowdhury. Effectively prefetching remote memory with Leap. In *Proc. USENIX ATC*, 2020. Far-memory prefetch heuristic; cited in §8.2 as an example of software-side swap optimisation.

[2] Emmanuel Amaro, Christopher Branner-Augmon, Zhihong Luo, Amy Ousterhout, Marcos K. Aguilera, Aurojit Panda, Sylvia Ratnasamy, and Scott Shenker. Can far memory improve job throughput? In *Proc. EuroSys*, 2020. Introduces Fastswap; reports $\sim$1 µs kernel-side overhead and batched-prefetch swap-in, the basis of the second swap profile in §8.2.

[3] Mina Tahmasbi Arashloo, Alexey Lavrov, Manya Ghobadi, Jennifer Rexford, David Walker, and David Wentzlaff. Enabling programmable transport protocols in high-speed NICs. In *Proc. USENIX NSDI*, 2020.

[4] Brian F. Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. Benchmarking cloud serving systems with YCSB. In *Proc. ACM SoCC*, 2010. The Yahoo! Cloud Serving Benchmark; we use the YCSB-A 50/50 Get-Put Zipfian workload in §8.3.

[5] CXL Consortium. Compute Express Link (CXL) Specification 3.1. `https://www.computeexpresslink.org/`, 2024.

[6] Aleksandar Dragojević, Dushyanth Narayanan, Orion Hodson, and Miguel Castro. FaRM: Fast remote memory. In *Proc. USENIX NSDI*, 2014.

[7] Haggai Eran, Lior Zeno, Maroun Tork, Gabi Malka, and Mark Silberstein. NICA: An infrastructure for inline acceleration of network applications. In *Proc. USENIX ATC*, 2019.

[8] Daniel Firestone, Andrew Putnam, Sambhrama Mundkur, Derek Chiou, Alireza Dabagh, Mike Andrewartha, Hari Angepat, et al. Azure Accelerated Networking: SmartNICs in the public cloud. In *Proc. USENIX NSDI*, 2018.

[9] Adithya Gangidi, Rui Miao, Shengbao Zheng, Sai Jayesh Bondu, Guilherme Goes, Hany Morsy, Rohit Puri, Mohammad Riftadi, Ashmitha Jeevaraj Shetty, Jingyi Yang, Shuqiang Zhang, Mikel Jimenez Fernandez, Shashidhar Gandham, and Hongyi Zeng. RDMA over Ethernet for distributed training at meta scale. In *Proc. ACM SIGCOMM*, 2024.

[10] Dan Gibson, Hema Hariharan, Eric Lance, Moray McLaren, Behnam Montazeri, Arjun Singh, Stephen Wang, Hassan M. G. Wassel, Zhehua Wu, Sunghwan Yoo, Raghuraman Balasubramanian, Prashant Chandra, Michael Cutforth, Peter Cuy, David Decotigny, Rakesh Gautam, Alex Iriza, Milo M. K. Martin, Rick Roy, Zuowei Shen, Ming Tan, Ye Tang, Monica Wong-Chan, Joe Zbiciak, and Amin Vahdat. Aquila: A unified, low-latency fabric for datacenter networks. In *Proc. USENIX NSDI*, 2022.

[11] Juncheng Gu, Youngmoon Lee, Yiwen Zhang, Mosharaf Chowdhury, and Kang G. Shin. Efficient memory disaggregation with Infiniswap. In *Proc. USENIX NSDI*, 2017. Kernel-side overhead of 3–5 µs on the swap-in path is the parameter referenced in §8.2.

[12] Zhiyuan Guo, Yizhou Shan, Xuhao Luo, Yutong Huang, and Yiying Zhang. Clio: A hardware-software co-designed disaggregated memory system. In *Proc. ACM ASPLOS*, 2022.

[13] Tingbo He. A time scaling theory for multi-layer electronic systems. *ChinaXiv*, May 2026. chinarxiv-202605.00224. Perspective from Huawei Semiconductor: τ scaling as successor to geometric Moore's-Law scaling; positions Unified Bus as the system-layer τ reduction mechanism with end-to-end remote-access latency from $\sim$10s of μs (TCP/IP-class) to $\sim$100 ns.

[14] Hongjing Huang, Jie Zhang, Xuzheng Chen, Ziyu Song, Jiajun Qin, and Zeke Wang. SwCC: Software-programmable and per-packet congestion control in RDMA engine. In *Proc. USENIX ATC*, 2025.

[15] Peihao Huang, Guo Chen, Xin Zhang, Can Liu, Hongyu Wang, Huijun Shen, Ying Bian, Yuanwei Lu, Zhenyuan Ruan, Bojie Li, Jiansong Zhang, Yongfeng Liu, and Zhigang Chen. Fast and scalable selective retransmission for RDMA. In *Proc. IEEE INFOCOM*, 2025.

[16] Peihao Huang, Xin Zhang, Zhigang Chen, Can Liu, and Guo Chen. LEFT: Lightweight and fast packet reordering for RDMA. In *Proc. APNet*, 2024.

[17] Huawei Technologies. UB-base-specification 2.0.1. `https://www.unifiedbus.org/`, 2024. Unified Bus consortium specification, available from the consortium's documentation portal.

[18] Huawei Technologies. Ascend 950 NPU architecture white paper. Huawei vendor white paper, May 2026. Architectural disclosure for the Ascend 950PR and 950DT NPUs; first publicly documented silicon implementing the Unified Bus spec, with URMA (asynchronous Write/Read/Send/Atomic via Jetty) and UB Memory (synchronous Load/Store + AtomicStore/Load/Swap/CAS) exposed as distinct on-die paths, plus URMA-CTP / URMA-TP transport modes, UBoE 2$\times$400 Gbps, a hardware Collective Communication Unit (CCU), UB On-Chip Switch for in-die forwarding, and a UB super-node target of 8192 cards (cluster target $>$128K).

[19] Stephen Ibanez, Alex Mallery, Serhat Arslan, Theo Jepsen, Muhammad Shahbaz, Nick McKeown, and Changhoon Kim. NanoTransport: A low-latency, programmable transport layer for NICs. In *Proc. ACM SOSR*, 2021.

[20] Anuj Kalia, Michael Kaminsky, and David G. Andersen. FaSST: Fast, scalable and simple distributed transactions with two-sided (RDMA) datagram RPCs. In *Proc. USENIX OSDI*, 2016.

[21] Anuj Kalia, Michael Kaminsky, and David G. Andersen. Datacenter RPCs can be general and fast. In *Proc. USENIX NSDI*, 2019.

[22] Antoine Kaufmann, Tim Stamler, Simon Peter, Naveen Kr. Sharma, Arvind Krishnamurthy, and Thomas Anderson. TAS: TCP acceleration as an OS service. In *Proc. EuroSys*, 2019. Reports detailed PCIe-class transaction latency decompositions used as parameter references in §5.

[23] Xinhao Kong, Jingrong Chen, Wei Bai, Yechen Xu, Mahmoud Elhaddad, Shachar Raindel, Jitendra Padhye, Alvin R. Lebeck, and Danyang Zhuo. Understanding RDMA microarchitecture resources for performance isolation. In *Proc. USENIX NSDI*, 2023.

[24] Xinhao Kong, Yibo Zhu, Huaping Zhou, Zhuo Jiang, Jianxi Ye, Chuanxiong Guo, and Danyang Zhuo. Collie: Finding performance anomalies in RDMA subsystems. In *Proc. USENIX NSDI*, 2022.

[25] Gautam Kumar, Nandita Dukkipati, Keon Jang, Hassan M. G. Wassel, Xian Wu, Behnam Montazeri, Yaogong Wang, Kevin Springborn, Christopher Alfeld, Michael Ryan, David Wetherall, and Amin Vahdat. Swift: Delay is simple and effective for congestion control in the datacenter. In *Proc. ACM SIGCOMM*, 2020.

[26] Yanfang Le, Rong Pan, Peter Newman, Jeremias Blendin, Abdul Kabbani, Vipin Jain, Raghava Sivaramu, and Francis Matus. STrack: A reliable multipath transport for AI/ML clusters. arXiv:2407.15266, 2024.

[27] Bojie Li. OpenClickNP: a clean-room reimplementation of ClickNP on Alveo U50. `https://github.com/bojieli/OpenClickNP`, 2025–2026.

[28] Bojie Li, Tianyi Cui, Zibo Wang, Wei Bai, and Lintao Zhang. SocksDirect: Datacenter sockets can be fast and compatible. In *Proc. ACM SIGCOMM*, 2019.

[29] Bojie Li, Zhenyuan Ruan, Wencong Xiao, Yuanwei Lu, Yongqiang Xiong, Andrew Putnam, Enhong Chen, and Lintao Zhang. KV-Direct: High-performance in-memory key-value store with programmable NIC. In *Proc. ACM SOSP*, 2017.

[30] Bojie Li, Kun Tan, Layong Larry Luo, Yanqing Peng, Renqian Luo, Ningyi Xu, Yongqiang Xiong, Peng Cheng, and Enhong Chen. ClickNP: Highly flexible and high-performance network processing with reconfigurable hardware. In *Proc. ACM SIGCOMM*, 2016.

[31] Bojie Li, Zhilong Xiang, Xiang Wang, Hongru Jonathan Zhou, and Kun Tan. FastWake: Revisiting host network stack for interrupt-mode RDMA. In *Proc. APNet*, 2023.

[32] Bojie Li, Gefei Zuo, Wei Bai, and Lintao Zhang. 1Pipe: Scalable total order communication in data center networks. In *Proc. ACM SIGCOMM*, 2021.

[33] Qiang Li, Yixiao Gao, Xiaoliang Wang, Haonan Qiu, Yanfang Le, Derui Liu, Qiao Xiang, Fei Feng, Peng Zhang, Bo Li, Jianbo Dong, Lingbo Tang, Hongqiang Harry Liu, Shaozong Liu, Weijie Li, Rui Miao, Yaohui Wu, Zhiwu Wu, Chao Han, Lei Yan, Zheng Cao, Zhongjie Wu, Chen Tian, Guihai Chen, Dennis Cai, Jinbo Wu, Jiaji Zhu, Jiesheng Wu, and Jiwu Shu. Flor: An open high performance RDMA framework over heterogeneous RNICs. In *Proc. USENIX OSDI*, 2023.

[34] Wenxue Li, Xiangzhou Liu, Yunxuan Zhang, Zihao Wang, Wei Gu, Tao Qian, Gaoxiong Zeng, Shoushou Ren, Xinyang Huang, Zhenghang Ren, Bowen Liu, Junxue Zhang, Kai Chen, and Bingyang Liu. Revisiting RDMA reliability for lossy fabrics. In *Proc. ACM SIGCOMM*, 2025. Best Student Paper, Honorable Mention.

[35] Yuliang Li, Rui Miao, Hongqiang Harry Liu, Yan Zhuang, Fei Feng, Lingbo Tang, Zheng Cao, Ming Zhang, Frank Kelly, Mohammad Alizadeh, and Minlan Yu. HPCC: High precision congestion control. In *Proc. ACM SIGCOMM*, 2019.

[36] Xiaofeng Lin, Yu Chen, Xiaodong Li, et al. Fastsocket: An almost drop-in replacement for the Linux socket interface for High-Performance Networking. In *Proc. USENIX ATC*, 2017.

[37] Jiaqi Lou, Xinhao Kong, Jinghan Huang, Wei Bai, Nam Sung Kim, and Danyang Zhuo. Harmonic: Hardware-assisted RDMA performance isolation for public clouds. In *Proc. USENIX NSDI*, 2024.

[38] Jason Lowe-Power et al. The gem5 simulator: Version 20.0+. arXiv:2007.03152, 2020. Open-source cycle-level micro-architecture simulator with SystemC TLM 2.0 interoperability bridge; v24.0.0.1 is used as the future-work substrate for full-system integration of `libopenurma_sc.a`.

[39] Yuanwei Lu, Guo Chen, Bojie Li, Kun Tan, Yongqiang Xiong, Peng Cheng, Jiansong Zhang, Enhong Chen, and Thomas Moscibroda. Memory efficient loss recovery for hardware-based transport in datacenter. In *Proc. APNet*, 2017.

[40] Yuanwei Lu, Guo Chen, Bojie Li, Kun Tan, Yongqiang Xiong, Peng Cheng, Jiansong Zhang, Enhong Chen, and Thomas Moscibroda. Multi-Path transport for RDMA in datacenters. In *Proc. USENIX NSDI*, 2018.

[41] Michael Marty, Marc de Kruijf, Jacob Adriaens, Christopher Alfeld, Sean Bauer, Carlo Contavalli, Michael Dalton, Nandita Dukkipati, William C. Evans, Steve Gribble, Nicholas Kidd, Roman Kononov, Gautam Kumar, Carl Mauer, Emily Musick, Lena Olson, Erik Rubow, Michael Ryan, Kevin Springborn, Paul Turner, Valas Valancius, Xi Wang, and Amin Vahdat. Snap: A microkernel approach to host networking. In *Proc. ACM SOSP*, 2019.

[42] Radhika Mittal, Vinh The Lam, Nandita Dukkipati, Emily Blem, Hassan Wassel, Monia Ghobadi, Amin Vahdat, Yaogong Wang, David Wetherall, and David Zats. TIMELY: RTT-based congestion control for the datacenter. In *Proc. ACM SIGCOMM*, 2015.

[43] Radhika Mittal, Alexander Shpiner, Aurojit Panda, Eitan Zahavi, Arvind Krishnamurthy, Sylvia Ratnasamy, and Scott Shenker. Revisiting network support for RDMA. In *Proc. ACM SIGCOMM*, 2018.

[44] NVIDIA Corporation. NVLink: A high-bandwidth inter-GPU interconnect. Vendor whitepaper, 2014–2025. Successive generations of the NVLink fabric are described in the NVIDIA whitepaper series.

[45] NVIDIA Corporation. NVIDIA BlueField-3 DPU datasheet. NVIDIA Networking product brief, 2023. Available from NVIDIA's data-processing-unit product page.

[46] NVIDIA Networking. NVIDIA Spectrum-X: Adaptive routing and telemetry-based congestion control for AI networks. NVIDIA technical brief, 2024. Vendor description of multi-path adaptive-routing

delivery over Spectrum-4 / BlueField-3 NICs; the closest commercially-deployed point of comparison to UB's TPG multi-path scheme.

[47] Yifan Qiao, Chenxi Wang, Zhenyuan Ruan, Adam Belay, Qingda Lu, Yiying Zhang, Miryung Kim, and Guoqing Harry Xu. Hermit: Low-latency, high-throughput, and transparent remote memory via feedback-directed asynchrony. In *Proc. USENIX NSDI*, 2023. Asynchronous remote-memory swap with feedback-directed I/O; cited in §8.2 for the same workload regime as Infiniswap/Fastswap.

[48] Sebastian Ramos and Torsten Hoefler. Designing high-performance, low-latency multi-cluster communication on modern InfiniBand networks. In *Proc. ACM HPDC*, 2023. Reports ConnectX-7 PCIe round-trip latencies in the $\sim$300–500 ns range; we use this as the parameterised PCIe RTT in §5.

[49] David Sidler, Zeke Wang, Monica Chiosa, Amit Kulkarni, and Gustavo Alonso. StRoM: Smart remote memory. In *Proc. EuroSys*, 2020.

[50] Arjun Singhvi, Aditya Akella, Dan Gibson, Thomas F. Wenisch, Monica Wong-Chan, Sean Clark, Milo M. K. Martin, Moray McLaren, Prashant Chandra, Rob Cauble, et al. 1RMA: Re-envisioning remote memory access for multi-tenant datacenters. In *Proc. ACM SIGCOMM*, 2020.

[51] Arjun Singhvi, Nandita Dukkipati, Prashant Chandra, Hassan M. G. Wassel, Naveen Kr. Sharma, Anthony Rebello, Henry Schuh, Praveen Kumar, Behnam Montazeri, Neelesh Bansod, Sarin Thomas, Inho Cho, Hyojeong Lee Seibert, Baijun Wu, Rui Yang, Yuliang Li, Kai Huang, Qianwen Yin, Abhishek Agarwal, Srinivas Vaduvatha, Weihuang Wang, Masoud Moshref, Tao Ji, David Wetherall, and Amin Vahdat. Falcon: A reliable, low latency hardware transport. In *Proc. ACM SIGCOMM*, 2025. Specification contributed to OCP 2024.

[52] Cha Hwan Song, Xin Zhe Khooi, Raj Joshi, Inho Choi, Jialin Li, and Mun Choon Chan. Network load balancing with in-network reordering support for RDMA. In *Proc. ACM SIGCOMM*, 2023.

[53] Ultra Ethernet Consortium. Ultra Ethernet specification 1.0. Industry specification, 2025. Released June 2025 under Linux Foundation JDF; `https://ultraethernet.org/`.

[54] Xizheng Wang, Guo Chen, Xijin Yin, Huichen Dai, Bojie Li, Binzhang Fu, and Kun Tan. StaR: Breaking the scalability limit for RDMA. In *Proc. IEEE ICNP*, 2021.

[55] Zilong Wang, Layong Luo, Qingsong Ning, Chaoliang Zeng, Wenxue Li, Xinchen Wan, Peng Xie, Tao Feng, Ke Cheng, Xiongfei Geng, Tianhao Wang, Weicheng Ling, Kejia Huo, Pingbo An, Kui Ji, Shideng Zhang, Bin Xu, Ruiqing Feng, Tao Ding, Kai Chen, and Chuanxiong Guo. SRNIC: A scalable architecture for RDMA NICs. In *Proc. USENIX NSDI*, 2023.

[56] Yiwen Zhang, Yue Tan, Brent Stephens, and Mosharaf Chowdhury. Justitia: Software multi-tenancy in hardware kernel-bypass networks. In *Proc. USENIX NSDI*, 2022.

[57] Chenxingyu Zhao, Jaehong Min, Ming Liu, and Arvind Krishnamurthy. White-boxing RDMA with packet-granular software control. In *Proc. USENIX NSDI*, 2025.

[58] Yibo Zhu, Haggai Eran, Daniel Firestone, Chuanxiong Guo, Marina Lipshteyn, Yehonatan Liron, Jitendra Padhye, Shachar Raindel, Mohamad Haj Yahia, and Ming Zhang. Congestion control for Large-Scale RDMA deployments. In *Proc. ACM SIGCOMM*, 2015.